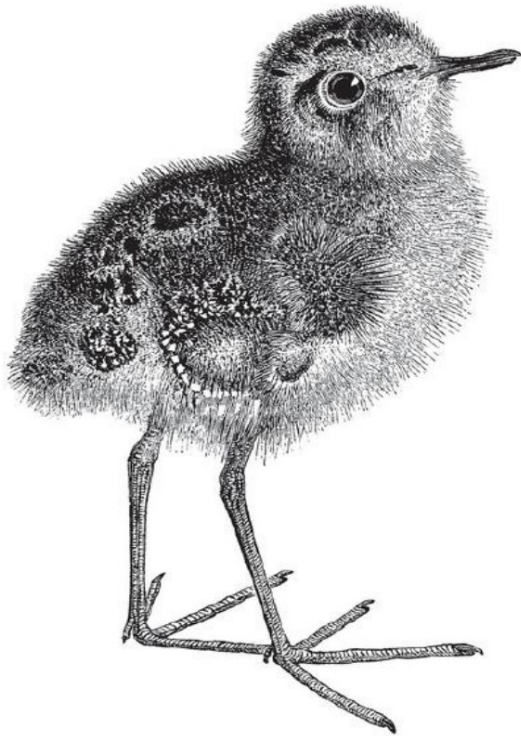


O'REILLY®

3rd Edition

Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow

Concepts, Tools, and Techniques
to Build Intelligent Systems



**Early
Release**
RAW &
UNEDITED

Aurélien Géron

Thực hành máy học với Scikit-Learn, Keras và... TensorFlow

ẤN BẢN THỨ BA

Khái niệm, công cụ và kỹ thuật để xây dựng trí tuệ nhân tạo
Hệ thống

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Aurélien Géron

OceanoPDF.com

Thực hành máy học với Scikit-Learn, Keras và...
TensorFlow

bởi Aurélien Géron

Bản quyền © 2022 Aurélien Géron. Mọi quyền được bảo lưu.

In tại Hoa Kỳ.

Xuất bản bởi O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol,
CA 95472.

Sách của O'Reilly có thể được mua để sử dụng cho mục đích giáo dục, kinh doanh
hoặc quảng bá bán hàng. Phiên bản trực tuyến cũng có sẵn cho hầu hết các đầu
sách (<http://oreilly.com>). Để biết thêm thông tin chi tiết, vui
lòng liên hệ bộ phận bán hàng doanh nghiệp/tổ chức của chúng tôi:
800-998-9938 hoặc corporate@oreilly.com.

- Biên tập viên phụ trách mua bản quyền: Nicole Butterfield
- Biên tập viên phát triển: Michele Cronin
- Biên tập viên sản xuất: Beth Kelly
- Biên tập viên: Kim Cofer
- Người hiệu đính:
- Người lập chỉ mục:
- Nhà thiết kế nội thất: David Futato
- Thiết kế bìa: Karen Montgomery
- Họa sĩ minh họa: Kate Dullea
- Tháng 10 năm 2022: Ấn bản thứ ba

Lịch sử sửa đổi cho phiên bản phát hành sớm

- 08/02/2022: Phát hành lần đầu
- 14/03/2022: Bản phát hành thứ hai
- 18/04/2022: Bản phát hành thứ ba
- 01/06/2022: Bản phát hành thứ tư

Xem <http://oreilly.com/catalog/errata.csp?isbn=9781098125974> Để biết thêm chi tiết về ngày phát hành.

Logo của O'Reilly là nhãn hiệu đã được đăng ký của O'Reilly Media, Inc. " Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow", hình ảnh bìa và các hình thức thương mại liên quan đều là nhãn hiệu của O'Reilly Media, Inc.

Các quan điểm được thể hiện trong tác phẩm này là của tác giả và không đại diện cho quan điểm của nhà xuất bản. Mặc dù nhà xuất bản và tác giả đã nỗ lực hết sức để đảm bảo rằng thông tin và hướng dẫn trong tác phẩm này là chính xác, nhà xuất bản và tác giả từ chối mọi trách nhiệm đối với các lỗi hoặc thiếu sót, bao gồm nhưng không giới hạn trách nhiệm đối với thiệt hại phát sinh từ việc sử dụng hoặc dựa vào tác phẩm này. Việc sử dụng thông tin và hướng dẫn trong tác phẩm này là rủi ro của riêng bạn. Nếu bất kỳ đoạn mã hoặc công nghệ nào khác mà tác phẩm này chứa đựng hoặc mô tả thuộc giấy phép mã nguồn mở hoặc quyền sở hữu trí tuệ của người khác, bạn có trách nhiệm đảm bảo rằng việc sử dụng chúng tuân thủ các giấy phép và/hoặc quyền đó.

978-1-098-12597-4

[LSI]

Lời tựa

Cơn sóng thần học máy

Năm 2006, Geoffrey Hinton và cộng sự đã công bố **một bài báo**¹. Bài viết này trình bày cách huấn luyện một mạng nơ-ron sâu có khả năng nhận dạng chữ số viết tay với độ chính xác hàng đầu (>98%). Họ đặt tên cho kỹ thuật này là "Học sâu" (Deep Learning). Mạng nơ-ron sâu là một mô hình (rất) đơn giản hóa của vỏ não chúng ta, bao gồm một chồng các lớp nơ-ron nhân tạo. Việc huấn luyện một mạng nơ-ron sâu được cho là bất khả thi vào thời điểm đó, và hầu hết các nhà nghiên cứu đã từ bỏ ý tưởng này vào cuối những năm 1990. Bài báo này đã khơi dậy sự quan tâm của cộng đồng khoa học, và chẳng bao lâu sau, nhiều bài báo mới đã chứng minh rằng Học sâu không chỉ khả thi mà còn có khả năng đạt được những thành tựu đáng kinh ngạc mà không một kỹ thuật Học máy (ML) nào khác có thể sánh kịp (với sự trợ giúp của sức mạnh tính toán khổng lồ và lượng dữ liệu lớn). Sự nhiệt tình này nhanh chóng lan rộng sang nhiều lĩnh vực khác của Học máy.

Một thập kỷ sau, Học máy đã chinh phục ngành công nghiệp và ngày nay nó là cốt lõi của phần lớn sự kỳ diệu trong các sản phẩm công nghệ cao, xếp hạng kết quả tìm kiếm trên web, hỗ trợ nhận dạng giọng nói trên điện thoại thông minh, đề xuất video và thậm chí có thể điều khiển ô tô của bạn.

Ứng dụng Học máy trong Dự án của Bạn

Vậy nên, chắc hẳn bạn rất hào hứng với Máy học và muốn tham gia vào lĩnh vực này!

Có lẽ bạn muốn trang bị cho con robot tự chế của mình một bộ não riêng?

Giúp nó nhận diện khuôn mặt? Hay học cách đi lại?

Hoặc có thể công ty của bạn có rất nhiều dữ liệu (nhật ký người dùng, dữ liệu tài chính, dữ liệu sản xuất, dữ liệu cảm biến máy móc, số liệu thống kê đường dây nóng, báo cáo nhân sự, v.v.) và

Rất có thể bạn sẽ tìm thấy một số viên ngọc quý tiềm ẩn nếu biết cách tìm kiếm. Với Học máy, bạn có thể thực hiện được những điều sau đây và nhiều hơn nữa:

- Phân khúc khách hàng và tìm ra chiến lược tiếp thị phù hợp nhất cho từng nhóm.
- Đề xuất sản phẩm cho từng khách hàng dựa trên những sản phẩm mà các khách hàng tương tự đã mua.
- Phát hiện những giao dịch nào có khả năng là gian lận.
- Dự báo doanh thu năm tới.

Dù lý do là gì, bạn đã quyết định học Máy học và áp dụng nó vào các dự án của mình. Thật là một ý tưởng tuyệt vời!

Mục tiêu và phương pháp tiếp cận

Cuốn sách này giả định rằng bạn hầu như không biết gì về Học máy. Mục tiêu của nó là cung cấp cho bạn các khái niệm, công cụ và trực giác cần thiết để triển khai các chương trình có khả năng học hỏi từ dữ liệu.

Chúng ta sẽ tìm hiểu rất nhiều kỹ thuật, từ những kỹ thuật đơn giản và phổ biến nhất (như Hồi quy tuyến tính) đến một số kỹ thuật Học sâu thường xuyên giành chiến thắng trong các cuộc thi. Để làm được điều này, chúng ta sẽ sử dụng các framework Python sẵn sàng cho môi trường sản xuất:

- **Scikit-Learn** Nó rất dễ sử dụng, nhưng lại triển khai nhiều thuật toán Học máy một cách hiệu quả, vì vậy nó là một điểm khởi đầu tuyệt vời để học Học máy. Nó được David Cournapeau tạo ra vào năm 2007, và hiện đang được dẫn dắt bởi một nhóm các nhà nghiên cứu tại Viện Nghiên cứu Khoa học Máy tính và Tự động hóa Pháp (Inria).
- **TensorFlow** là một thư viện phức tạp hơn dành cho tính toán số phân tán. Nó cho phép huấn luyện và chạy các mạng nơ-ron rất lớn một cách hiệu quả bằng cách phân phối các phép tính trên nhiều máy tính khác nhau.

Hàng trăm máy chủ đa GPU (bộ xử lý đồ họa). TensorFlow (TF) được tạo ra tại Google và hỗ trợ nhiều ứng dụng Học máy quy mô lớn của công ty. Nó được công khai mã nguồn vào tháng 11 năm 2015, và phiên bản 2.0 được phát hành vào tháng 9 năm 2019.

- **Keras** Keras là một API học sâu cấp cao giúp việc huấn luyện và chạy mạng nơ-ron trở nên rất đơn giản. Keras được tích hợp sẵn TensorFlow và dựa vào TensorFlow cho tất cả các phép tính chuyên sâu.

Cuốn sách ưu tiên phương pháp thực hành, giúp người đọc hiểu sâu sắc về Học máy thông qua các ví dụ cụ thể và một chút lý thuyết.

MẸO

Mặc dù bạn có thể đọc cuốn sách này mà không cần dùng đến máy tính xách tay, tôi vẫn đặc biệt khuyên bạn nên thử nghiệm với các ví dụ mã lập trình.

Ví dụ mã

Tất cả các ví dụ mã trong cuốn sách này đều là mã nguồn mở và có sẵn trực tuyến tại <https://github.com/ageron/handson-ml3>, Chúng được gọi là Jupyter Notebook. Đây là những tài liệu tương tác chứa văn bản, hình ảnh và các đoạn mã có thể thực thi (trong trường hợp này là Python). Cách dễ nhất và nhanh nhất để bắt đầu là chạy các notebook này bằng Google Colab: đây là một dịch vụ miễn phí cho phép bạn chạy bất kỳ Jupyter Notebook nào trực tiếp trực tuyến, mà không cần phải cài đặt bất cứ thứ gì trên máy tính của bạn. Tất cả những gì bạn cần là một trình duyệt web và một tài khoản Google.

GHI CHÚ

Trong cuốn sách này, tôi giả định rằng bạn đang sử dụng Google Colab, nhưng tôi cũng đã thử nghiệm các notebook trên các nền tảng trực tuyến khác như Kaggle hoặc Binder, vì vậy bạn có thể sử dụng chúng nếu muốn. Ngoài ra, bạn có thể cài đặt các thư viện và công cụ cần thiết (hoặc ảnh Docker cho cuốn sách này) và chạy trực tiếp các notebook trên máy tính của mình. Xem hướng dẫn tại <https://homl.info/install>.

Điều kiện tiên quyết

Cuốn sách này giả định rằng bạn đã có một số kinh nghiệm lập trình Python. Nếu bạn chưa biết Python, **hãy truy cập <https://learnpython.org/>** Đây là một điểm khởi đầu tuyệt vời. Hướng dẫn chính thức trên **Python.org** Cũng khá tốt.

Cuốn sách này cũng giả định rằng bạn đã quen thuộc với các thư viện khoa học chính của Python—đặc biệt là **NumPy**, **Gấu trúc**, và **Matplotlib**. Nếu bạn chưa từng sử dụng các thư viện này, đừng lo lắng, chúng rất dễ học và tôi đã tạo hướng dẫn cho từng thư viện. Bạn có thể truy cập chúng trực tuyến tại **<https://homl.info/tutorials>**.

Hơn nữa, nếu bạn muốn hiểu đầy đủ cách thức hoạt động của các thuật toán Học máy (không chỉ cách sử dụng chúng), thì bạn cần phải có ít nhất kiến thức cơ bản về một vài khái niệm toán học, đặc biệt là đại số tuyến tính.

Cụ thể, bạn cần biết vectơ và ma trận là gì, và cách thực hiện một số phép toán đơn giản như cộng vectơ, hoặc chuyển vị và nhân ma trận. Nếu bạn cần một phần giới thiệu nhanh về đại số tuyến tính (thực ra nó không quá khó!), tôi đã tạo một hướng dẫn tại **<https://homl.info/tutorials>**. Bạn cũng sẽ tìm thấy hướng dẫn về phép tính vi phân, có thể hữu ích để hiểu cách huấn luyện mạng nơ-ron, nhưng không hoàn toàn cần thiết để nắm bắt các khái niệm quan trọng. Cuốn sách này cũng thỉnh thoảng sử dụng các khái niệm toán học khác, chẳng hạn như hàm mũ và logarit, một chút lý thuyết xác suất và một số khái niệm thống kê cơ bản, nhưng không quá nâng cao. Nếu bạn cần trợ giúp về bất kỳ điều nào trong số này, vui lòng truy cập **<https://khanacademy.org/>**, nơi cung cấp nhiều khóa học toán trực tuyến miễn phí và chất lượng cao.

Lộ trình

Cuốn sách này được chia thành hai phần. **Phần I, “Những nguyên lý cơ bản của máy học”**, bao gồm các chủ đề sau:

- Máy học là gì, nó cố gắng giải quyết những vấn đề gì, và các loại chính cũng như các khái niệm cơ bản của hệ thống máy học.

- Các bước trong một dự án Học máy điển hình
- Học hỏi bằng cách điều chỉnh mô hình cho phù hợp với dữ liệu.
- Tối ưu hóa hàm chi phí
- Xử lý, làm sạch và chuẩn bị dữ liệu
- Lựa chọn và thiết kế các tính năng
- Lựa chọn mô hình và tinh chỉnh siêu tham số bằng phương pháp kiểm định chéo.
- Những thách thức của Học máy, đặc biệt là hiện tượng thiếu khớp (underfitting) và quá khớp (overfitting) (sự đánh đổi giữa độ lệch và phương sai).
- Các thuật toán học máy phổ biến nhất: Tuyến tính và Đa thức
Hồi quy, Hồi quy Logistic, Thuật toán k-Hàng xóm gần nhất, Thuật toán vectơ hỗ trợ
Máy học, cây quyết định, rừng ngẫu nhiên và các phương pháp kết hợp
- Giảm số chiều của dữ liệu huấn luyện để chống lại "lời nguyền về số chiều"
- Các kỹ thuật học không giám sát khác bao gồm phân cụm, ước lượng mật độ và phát hiện bất thường.

Phần II, "Mạng nơ-ron và học sâu", bao gồm các chủ đề sau:

- Mạng nơ-ron là gì và chúng có ích lợi gì?
- Xây dựng và huấn luyện mạng nơ-ron bằng TensorFlow và Keras
- Các kiến trúc mạng nơ-ron quan trọng nhất: mạng nơ-ron truyền thẳng cho dữ liệu dạng bảng, mạng tích chập cho thị giác máy tính, mạng nơ-ron hồi quy và mạng bộ nhớ dài hạn ngắn hạn (LSTM) cho xử lý chuỗi, bộ mã hóa/giải mã và Transformer cho xử lý ngôn ngữ tự nhiên (và nhiều hơn nữa!), bộ tự mã hóa, mạng đối kháng tạo sinh (GAN) và mô hình khuếch tán cho học tạo sinh.
- Các kỹ thuật huấn luyện mạng nơ-ron sâu

- Làm thế nào để xây dựng một tác nhân (ví dụ: một bot trong trò chơi) có thể học các chiến lược tốt thông qua thử và sai, sử dụng Học tăng cường (Reinforcement Learning)
- Tải và xử lý sơ bộ lượng lớn dữ liệu một cách hiệu quả
- Huấn luyện và triển khai các mô hình TensorFlow ở quy mô lớn

Phần đầu chủ yếu dựa trên Scikit-Learn, trong khi phần thứ hai sử dụng TensorFlow và Keras.

THẬN TRỌNG

Đứng vững vàng lao vào vùng nước sâu: Mặc dù Học sâu (Deep Learning) chắc chắn là một trong những lĩnh vực thú vị nhất trong Học máy (Machine Learning), bạn vẫn nên nắm vững các nguyên tắc cơ bản trước. Hơn nữa, hầu hết các vấn đề có thể được giải quyết khá tốt bằng các kỹ thuật đơn giản hơn như Rừng ngẫu nhiên và các phương pháp kết hợp (đã thảo luận trong [Phần I](#)). Học sâu phù hợp nhất cho các vấn đề phức tạp như nhận dạng hình ảnh, nhận dạng giọng nói hoặc xử lý ngôn ngữ tự nhiên, và nó đòi hỏi rất nhiều dữ liệu, sức mạnh tính toán và sự kiên nhẫn (trừ khi bạn có thể tận dụng mạng nơ-ron được huấn luyện trước, như chúng ta sẽ thấy).

Những thay đổi giữa phần đầu tiên và phần thứ hai

Phiên bản

Nếu bạn đã đọc ấn bản đầu tiên, đây là những thay đổi chính giữa ấn bản đầu tiên và ấn bản thứ hai:

- Toàn bộ mã nguồn đã được chuyển đổi từ TensorFlow 1.x sang TensorFlow 2.x, và hầu hết mã cấp thấp của TensorFlow (đồ thị, phiên, cột đặc trưng, Estimators, v.v.) đã được thay thế bằng mã Keras đơn giản hơn nhiều.
 - Phiên bản thứ hai cũng giới thiệu API Dữ liệu để tải và xử lý trước các tập dữ liệu lớn, API Chiến lược Phân phối để huấn luyện và triển khai các mô hình TF ở quy mô lớn, TF Serving và Google Cloud AI.
- Nền tảng để đưa các mô hình vào sản xuất, và nó đã giới thiệu ngắn gọn về TF.

Transform, TFLite, TF Addons/Seq2Seq, TensorFlow.js và TF Agents.

- Nó cũng giới thiệu nhiều chủ đề học máy bổ sung, bao gồm một chương mới về học không giám sát, các kỹ thuật thị giác máy tính để phát hiện đối tượng và phân đoạn ngữ nghĩa, xử lý chuỗi bằng mạng nơ-ron tích chập (CNN), xử lý ngôn ngữ tự nhiên (NLP) bằng mạng nơ-ron hồi quy (RNN), CNN và Transformer, mạng đối kháng tạo sinh (GAN), và nhiều hơn nữa.

Xem <https://homl.info/changes2> Để biết thêm chi tiết.

Những thay đổi giữa phần thứ hai và phần thứ ba

Phiên bản

Nếu bạn đọc ấn bản thứ hai, đây là những thay đổi chính giữa ấn bản thứ hai và ấn bản thứ ba:

- Toàn bộ mã nguồn đã được cập nhật lên phiên bản thư viện mới nhất. Đặc biệt, ấn bản thứ ba này giới thiệu nhiều tính năng mới cho Scikit-Learn (ví dụ: theo dõi tên đặc trưng, Gradient Boosting dựa trên Histogram, lan truyền nhãn, v.v.). Nó cũng giới thiệu thư viện Keras Tuner để tinh chỉnh siêu tham số, thư viện transformers của Hugging Face cho xử lý ngôn ngữ tự nhiên, và các lớp tiền xử lý và tăng cường dữ liệu mới của Keras.
- Một số mô hình thị giác đã được thêm vào (ResNeXt, DenseNet, MobileNet, CSPNet và EfficientNet), cũng như các hướng dẫn để lựa chọn đúng một.
- **Chương 15** giờ đây phân tích dữ liệu về lượng hành khách đi xe buýt và đường sắt ở Chicago thay vì chuỗi thời gian được tạo ra, và giới thiệu mô hình ARMA cùng các biến thể của nó.
- **Chương 16** về xử lý ngôn ngữ tự nhiên hiện đang xây dựng một bộ dịch từ tiếng Anh sang... Mô hình dịch tiếng Tây Ban Nha, ban đầu sử dụng mạng RNN mã hóa-giải mã, sau đó

Sử dụng mô hình Transformer. Chương này cũng bao gồm các mô hình ngôn ngữ bổ sung, chẳng hạn như Switch Transformers, DistilBERT, T5 và PaLM (với Chain-of-Thought Prompting). Nó cũng giới thiệu Visual Transformers (ViTs) và cung cấp tổng quan về một vài mô hình hình ảnh dựa trên Transformer, chẳng hạn như DETection TRansformer (DETR), Perceiver, DINO, cũng như tổng quan ngắn gọn về một số mô hình đa phương thức lớn, bao gồm CLIP, DALL·E, Flamingo và GATO.

- **Chương 17** về học tạo sinh giới thiệu các mô hình khuếch tán và triển khai Mô hình Xác suất Khuếch tán Khử nhiễu (DDPM) từ đầu.
- **Chương 19** đã được chuyển từ Google AI Platform sang Google Vertex AI và sử dụng Keras Tuner phân tán để tìm kiếm siêu tham số quy mô lớn. Chương này hiện bao gồm mã TensorFlow.js mà bạn có thể thử nghiệm trực tuyến. Nó cũng giới thiệu thêm các kỹ thuật huấn luyện phân tán, bao gồm Pipedream và Pathways.
- Để đáp ứng nhu cầu về nội dung mới, một số phần đã được chuyển lên trực tuyến, bao gồm hướng dẫn cài đặt, Kernel PCA, chi tiết toán học của Bayesian Gaussian Mixtures, TF Agents, và các phụ lục cũ A (giải bài tập), C (toán học SVM) và E (kiến trúc mạng nơ-ron bổ sung).

Xem <https://homl.info/changes3> để biết thêm chi tiết.

Các nguồn tài liệu khác

Có rất nhiều nguồn tài liệu tuyệt vời để học về Học máy.

Ví dụ, **khóa học ML của Andrew Ng trên Coursera**. Thật tuyệt vời, mặc dù nó đòi hỏi một khoản đầu tư thời gian đáng kể.

Ngoài ra còn có rất nhiều trang web thú vị về Học máy, bao gồm cả **Hướng dẫn sử dụng xuất sắc của Scikit-Learn**. Bạn cũng có thể thích **Dataquest**, nơi cung cấp các bài hướng dẫn tương tác rất hay, và các blog về học máy như những blog được liệt kê trên **Quora**.

Có rất nhiều sách nhập môn khác về Học máy. Cụ thể là:

- Khoa học dữ liệu từ cơ bản của Joel Grus , Phiên bản thứ 2 (O'Reilly) trình bày những kiến thức cơ bản về Học máy và triển khai một số thuật toán chính bằng ngôn ngữ Python thuần túy (từ đầu, như tên gọi cho thấy).
- Cuốn sách "Machine Learning: An Algorithmic Perspective, 2nd edition" của Stephen Marsland (Chapman & Hall) là một tài liệu giới thiệu tuyệt vời về Học máy, bao gồm nhiều chủ đề chuyên sâu với các ví dụ mã bằng Python (cũng được viết từ đầu, nhưng sử dụng NumPy).
- Cuốn sách Python Machine Learning, ấn bản thứ 3 của Sebastian Raschka (Packt Publishing) cũng là một tài liệu giới thiệu tuyệt vời về Machine Learning và tận dụng các thư viện mã nguồn mở của Python (Pylearn 2 và Theano).
- Cuốn sách "Deep Learning with Python" của François Chollet , ấn bản lần 2 (Manning), là một cuốn sách rất thực tiễn, bao quát nhiều chủ đề một cách rõ ràng và súc tích, đúng như mong đợi từ tác giả của thư viện Keras xuất sắc. Sách chú trọng vào các ví dụ mã lập trình hơn là lý thuyết toán học.
- Cuốn sách "The Hundred-Page Machine Learning Book" của Andriy Burkov rất ngắn gọn nhưng bao quát một phạm vi chủ đề ấn tượng, giới thiệu chúng bằng ngôn ngữ dễ hiểu mà không hề né tránh các phương trình toán học.
- Cuốn sách Learning from Data (AMLBook) của Yaser S. Abu-Mostafa, Malik Magdon-Ismail và Hsuan-Tien Lin là một cách tiếp cận khá lý thuyết đối với học máy, cung cấp những hiểu biết sâu sắc, đặc biệt là về sự đánh đổi giữa độ lệch và phương sai (xem [Chương 4](#)).
- Cuốn sách "Trí tuệ nhân tạo: Một cách tiếp cận hiện đại", ấn bản thứ 4 (Pearson) của Stuart Russell và Peter Norvig là một tác phẩm tuyệt vời (và đồ sộ) bao quát một lượng lớn các chủ đề, trong đó có Học máy. Nó giúp chúng ta hiểu rõ hơn về Học máy.
- Cuốn sách Deep Learning for Coders with fastai and PyTorch của Jeremy Howard và Sylvain Gugger (O'Reilly) cung cấp một cách giải thích vô cùng rõ ràng và dễ hiểu.

Giới thiệu thực tiễn về Học sâu (Deep Learning) sử dụng thư viện fastai và PyTorch.

Cuối cùng, tham gia các trang web tổ chức cuộc thi học máy như [Kaggle.com](https://www.kaggle.com). Điều này sẽ cho phép bạn thực hành các kỹ năng của mình trên các vấn đề thực tế, với sự trợ giúp và những hiểu biết sâu sắc từ một số chuyên gia ML hàng đầu hiện nay.

Các quy ước được sử dụng trong cuốn sách này

Cuốn sách này sử dụng các quy ước in ấn sau:

Chữ nghiêng

Chỉ định các thuật ngữ, URL, địa chỉ email, tên tệp và phần mở rộng tệp mới.

Chiều rộng không đổi

Được sử dụng trong danh sách chương trình, cũng như trong các đoạn văn để tham chiếu đến các thành phần của chương trình như tên biến hoặc hàm, cơ sở dữ liệu, kiểu dữ liệu, biến môi trường, câu lệnh và từ khóa.

Chữ đậm với độ rộng không đổi

Hiển thị các lệnh hoặc văn bản khác mà người dùng cần nhập chính xác.

Chữ nghiêng có độ rộng không đổi

Hiển thị văn bản cần được thay thế bằng các giá trị do người dùng cung cấp hoặc bằng các giá trị được xác định bởi ngữ cảnh.

MẸO

Yếu tố này biểu thị một lời khuyên hoặc gợi ý.

GHI CHÚ

Yếu tố này biểu thị một ghi chú chung.

CẢNH BÁO

Yếu tố này biểu thị một lời cảnh báo hoặc lưu ý.

Học trực tuyến O'Reilly

GHI CHÚ

Trong hơn 40 năm qua, **O'Reilly Media** đã cung cấp đào tạo về công nghệ và kinh doanh, kiến thức và hiểu biết sâu sắc để giúp các công ty thành công.

Mạng lưới chuyên gia và nhà sáng tạo độc đáo của chúng tôi chia sẻ kiến thức và chuyên môn của họ thông qua sách, bài báo và nền tảng học trực tuyến của chúng tôi. Nền tảng học trực tuyến của O'Reilly cung cấp cho bạn quyền truy cập theo yêu cầu vào các khóa đào tạo trực tiếp, lộ trình học tập chuyên sâu, môi trường lập trình tương tác và một bộ sưu tập khổng lồ các tài liệu văn bản và video từ O'Reilly và hơn 200 nhà xuất bản khác. Để biết thêm thông tin, hãy truy cập <https://oreilly.com>.

Cách liên hệ với chúng tôi

Vui lòng gửi ý kiến và câu hỏi liên quan đến cuốn sách này đến nhà xuất bản:

- Công ty truyền thông O'Reilly
- 1005 Đường Gravenstein phía Bắc

- Sebastopol, CA 95472
- 800-998-9938 (tại Hoa Kỳ hoặc Canada)
- 707-829-0515 (quốc tế hoặc nội địa)
- 707-829-0104 (fax)

Chúng tôi có một trang web dành cho cuốn sách này, nơi chúng tôi liệt kê các lỗi sai, ví dụ và bất kỳ thông tin bổ sung nào khác. Bạn có thể truy cập trang này tại <https://homl.info/oreilly2>.

Gửi email đến bookquestions@oreilly.com để bình luận hoặc đặt câu hỏi kỹ thuật về cuốn sách này.

Để biết thêm tin tức và thông tin về sách và các khóa học của chúng tôi, hãy truy cập <https://oreilly.com>.

Tìm chúng tôi trên LinkedIn: <https://linkedin.com/company/oreilly-media>

Theo dõi chúng tôi trên Twitter: <https://twitter.com/oreillymedia>

Hãy xem chúng tôi trên YouTube: <https://www.youtube.com/oreillymedia>

Lời cảm ơn

Tôi chưa bao giờ tưởng tượng rằng ấn bản đầu tiên và thứ hai của cuốn sách này lại nhận được sự đón nhận lớn đến vậy. Tôi đã nhận được rất nhiều tin nhắn từ độc giả, nhiều người đặt câu hỏi, một số người tốt bụng chỉ ra lỗi sai, và hầu hết đều gửi cho tôi những lời động viên. Tôi không thể diễn tả hết lòng biết ơn của mình đối với tất cả các độc giả vì sự ủng hộ to lớn của họ. Cảm ơn tất cả các bạn rất nhiều!

Vui lòng đừng ngần ngại **báo cáo sự cố trên GitHub**. Nếu bạn phát hiện lỗi trong các ví dụ mã (hoặc chỉ để hỏi), hoặc để gửi **báo cáo lỗi**, hãy liên hệ với chúng tôi. Nếu bạn phát hiện lỗi trong văn bản, hãy cho chúng tôi biết. Một số độc giả cũng chia sẻ cách cuốn sách này giúp họ có được công việc đầu tiên, hoặc giúp họ giải quyết một vấn đề cụ thể mà họ đang gặp phải. Tôi thấy những phản hồi như vậy vô cùng có tính động viên. Nếu bạn thấy cuốn sách này hữu ích, tôi rất mong bạn có thể chia sẻ câu chuyện của mình với tôi, dù là riêng tư (ví dụ:

(qua [LinkedIn](#)) hoặc công khai (ví dụ: tweet cho tôi tại @aureliengeron hoặc viết đánh giá trên Amazon).

Tôi cũng xin gửi lời cảm ơn sâu sắc đến tất cả những người tuyệt vời đã dành thời gian và chuyên môn của mình để xem xét lại ấn bản thứ ba này, sửa lỗi và đưa ra vô số gợi ý. Ấn bản này đã tốt hơn rất nhiều nhờ họ: Olzhas Akpambetov, George Bonner, François Chollet, Siddha Gangju, Sam Goodman, Matt Harrison, Sasha Sobran, Lewis Tunstall, Leandro von Werra, và người anh em thân yêu của tôi, Sylvain. Tất cả các bạn đều tuyệt vời!

Tôi cũng vô cùng biết ơn nhiều người đã hỗ trợ tôi trong suốt quá trình, bằng cách trả lời các câu hỏi của tôi, đề xuất cải tiến và đóng góp vào mã nguồn trên Github, đặc biệt là Yannick Assogba, Ian Beauregard, Ulf Bissbort, Rick Chao, Peretz Cohen, Kyle Gallatin, Hannes Hapke, Victor Khaustov, Soonson Kwon, Eric Lebigot, Jason Mayes, Laurence Moroney, Sara Robinson, Joaquín Ruales và Yuefeng Zhou.

Cuốn sách này sẽ không thể tồn tại nếu không có đội ngũ nhân viên tuyệt vời của O'Reilly, đặc biệt là Nicole Taché, người đã cho tôi những phản hồi sâu sắc và luôn vui vẻ, động viên và giúp đỡ: Tôi không thể mơ ước một biên tập viên nào tốt hơn. Cảm ơn rất nhiều Michele Cronin, người đã cổ vũ tôi trong những chương cuối cùng và giúp tôi hoàn thành cuốn sách. Cảm ơn Kristen Brown, biên tập viên sản xuất cho ấn bản thứ hai và thứ ba, người đã giám sát toàn bộ quá trình (cô ấy cũng điều phối việc sửa chữa và cập nhật cho mỗi lần tái bản của ấn bản đầu tiên và thứ hai). Cảm ơn Rachel Monaghan và Amanda Kersey vì đã biên tập bản thảo kỹ lưỡng, và Johnny O'Toole, người đã quản lý mối quan hệ với Amazon và trả lời nhiều câu hỏi của tôi. Cảm ơn Marie Beaugureau, Ben Loric, Mike Loukides và Laurel Ruma vì đã tin tưởng vào dự án này và giúp tôi xác định phạm vi của nó.

Xin cảm ơn Matt Hacker và toàn thể nhóm Atlas đã giải đáp mọi thắc mắc kỹ thuật của tôi về định dạng, AsciiDoc, MathML và LaTeX, và cảm ơn Nick Adams, Rebecca Demarest, Rachel Head, Judith McConville, Helen Monroe, Karen Montgomery, Rachel Roumeliotis và tất cả những người khác tại O'Reilly đã đóng góp cho cuốn sách này.

Tôi sẽ không bao giờ quên tất cả những người tuyệt vời đã giúp đỡ tôi trong hai ấn bản đầu tiên của cuốn sách này: bạn bè, đồng nghiệp, chuyên gia, bao gồm nhiều thành viên của nhóm TensorFlow, danh sách rất dài: Olzhas Akpambetov, Karmel Allison, Martin Andrews, David Andrzejewski, Paige Bailey, Lukas Biewald, Eugene Brevdo, William Chargin, François Chollet, Clément Courbet, Robert Crowe, Mark Daoust, Daniel “Wolff” Dobson, Julien Dubois, Mathias Kende, Daniel Kitachewsky, Nick Felt, Bruce Fontaine, Justin Francis, Goldie Gadde, Irene Giannoumis, Ingrid von Glehn, Vincent Guilbeau, Sandeep Gupta, Priya Gupta, Kevin Haas, Eddy Hung, Konstantinos Katsiapis, Viacheslav Kovalevskyi, Jon Krohn, Allen Lavoie, Karim Matrah, Grégoire Mesnil, Clemens Mewald, Dan Moldovan, Dominic Monn, Sean Morgan, Tom O'Malley, James Pack, Alexander Pak, Haesun Park, Alexandre Passos, Ankur Patel, Josh Patterson, André Susano Pinto, Anthony Platanios, Anosh Raj, Oscar Ramirez, Anna Revinskaya, Saurabh Saxena, Salim Sémaoune, Ryan Sepassi, Vitor Sessak, Jiri Simsa, Iain Smears, Xiaodan Song, Christina Sorokin, Michel Tessier, Wiktor Tomczak, Dustin Tran, Todd Wang, Pete Warden, Rich Washington, Martin Wicke, Edd Wilder-James, Sam Witteveen, Jason Zaman, Yuefeng Zhou, và anh trai tôi Sylvain.

Cuối cùng nhưng không kém phần quan trọng, tôi vô cùng biết ơn người vợ yêu quý của mình, Emmanuelle, và ba đứa con tuyệt vời của chúng tôi, Alexandre, Rémi và Gabrielle, vì đã động viên tôi nỗ lực hết mình cho cuốn sách này. Sự tò mò không ngừng nghỉ của các con là vô giá: việc giải thích một số khái niệm khó hiểu nhất trong cuốn sách này cho vợ con đã giúp tôi làm rõ suy nghĩ và trực tiếp cải thiện nhiều phần của nó. Thêm vào đó, các con luôn mang bánh quy và cà phê đến cho tôi, còn gì tuyệt vời hơn nữa?

¹ Geoffrey E. Hinton et al., “Một thuật toán học nhanh cho mạng niềm tin sâu,” *Tính toán thần kinh* 18 (2006): 1527-1554.

² Mặc dù mạng nơ-ron tích chập sâu của Yann LeCun đã hoạt động tốt trong Nhận dạng hình ảnh đã xuất hiện từ những năm 1990, mặc dù chúng không phải là loại đa năng.

Phần I. Những nguyên lý cơ bản của máy học

OceanofPDF.com

Chương 1. Bức tranh tổng quan về học máy

LƯU Ý DÀNH CHO ĐỘC GIẢ ĐẶT MUA SỚM

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương 1 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Cách đây không lâu, nếu bạn nhấc điện thoại lên và hỏi đường về nhà, nó sẽ phớt lờ bạn - và mọi người sẽ nghi ngờ sự tỉnh táo của bạn. Nhưng Học máy (Machine Learning) không còn là khoa học viễn tưởng nữa: hàng tỷ người sử dụng nó mỗi ngày. Và sự thật là nó đã tồn tại hàng thập kỷ trong một số ứng dụng chuyên biệt, chẳng hạn như Nhận dạng ký tự quang học (OCR). Ứng dụng Học máy đầu tiên thực sự trở nên phổ biến, cải thiện cuộc sống của hàng trăm triệu người, đã chiếm lĩnh thế giới vào những năm 1990: bộ lọc thư rác. Nó không hẳn là một robot tự nhận thức, nhưng về mặt kỹ thuật, nó đủ điều kiện là Học máy: nó đã học tốt đến mức bạn hiếm khi cần phải đánh dấu email là thư rác nữa. Sau đó là hàng trăm ứng dụng Học máy hiện đang âm thầm hỗ trợ hàng trăm sản phẩm và tính năng mà bạn sử dụng thường xuyên: lời nhắc bằng giọng nói, dịch tự động, tìm kiếm hình ảnh, đề xuất sản phẩm, và nhiều hơn nữa.

Máy học bắt đầu từ đâu và kết thúc ở đâu? Chính xác thì việc một cỗ máy học được điều gì đó có nghĩa là gì? Nếu tôi tải xuống bản sao của tất cả các bài viết trên Wikipedia, liệu máy tính của tôi có thực sự học được điều gì đó không? Liệu nó có đột nhiên thông minh hơn không? Trong chương này, tôi sẽ bắt đầu bằng cách làm rõ Máy học là gì và tại sao bạn có thể muốn sử dụng nó.

Trước khi bắt đầu khám phá "lục địa" Học máy, chúng ta sẽ xem xét bản đồ và tìm hiểu về các khu vực chính cũng như những địa danh nổi bật: học có giám sát so với học không giám sát và các biến thể của chúng, học trực tuyến so với học theo lô, học dựa trên trường hợp so với học dựa trên mô hình. Sau đó, chúng ta sẽ xem xét quy trình làm việc của một dự án Học máy điển hình, thảo luận về những thách thức chính mà bạn có thể gặp phải, và tìm hiểu cách đánh giá và tinh chỉnh một hệ thống Học máy.

Chương này giới thiệu rất nhiều khái niệm cơ bản (và thuật ngữ chuyên ngành) mà mọi nhà khoa học dữ liệu nên nắm vững. Đây sẽ là một cái nhìn tổng quan cấp cao (đây là chương duy nhất không có nhiều mã lập trình), tất cả đều khá đơn giản, nhưng mục tiêu của tôi là đảm bảo mọi thứ đều rõ ràng với bạn trước khi chúng ta tiếp tục sang phần còn lại của cuốn sách. Vậy thì hãy pha một tách cà phê và cùng bắt đầu nào!

MẸO

Nếu bạn đã quen thuộc với những kiến thức cơ bản về Học máy, bạn có thể bỏ qua thẳng đến [Chương 2](#). Nếu bạn chưa chắc chắn, hãy cố gắng trả lời tất cả các câu hỏi được liệt kê ở cuối chương trước khi chuyển sang chương tiếp theo.

Máy học là gì?

Máy học là khoa học (và nghệ thuật) lập trình máy tính để chúng có thể học hỏi từ dữ liệu.

Dưới đây là một định nghĩa tổng quát hơn một chút:

[Máy học] là lĩnh vực nghiên cứu giúp máy tính có khả năng học hỏi mà không cần được lập trình rõ ràng.

—Arthur Samuel, 1959

Và một phiên bản thiên về kỹ thuật hơn:

Một chương trình máy tính được cho là học hỏi từ kinh nghiệm E liên quan đến một nhiệm vụ T và một thước đo hiệu suất P, nếu hiệu suất của nó trên nhiệm vụ T, được đo bằng P, cải thiện theo kinh nghiệm E.

—Tom Mitchell, 1997

Bộ lọc thư rác của bạn là một chương trình Học máy, dựa trên các ví dụ về email rác (được người dùng gắn cờ) và các ví dụ về email thông thường (không phải thư rác, còn gọi là "ham"), có thể học cách gắn cờ thư rác. Các ví dụ mà hệ thống sử dụng để học được gọi là tập huấn luyện. Mỗi ví dụ huấn luyện được gọi là một trường hợp huấn luyện (hoặc mẫu). Phần của hệ thống Học máy học và đưa ra dự đoán được gọi là mô hình. Mạng nơ-ron và rừng ngẫu nhiên là những ví dụ về mô hình.

Trong trường hợp này, nhiệm vụ T là gắn cờ thư rác cho các email mới, kinh nghiệm E là dữ liệu huấn luyện, và thước đo hiệu suất P cần được xác định; ví dụ, bạn có thể sử dụng tỷ lệ email được phân loại chính xác. Thước đo hiệu suất cụ thể này được gọi là độ chính xác, và nó thường được sử dụng trong các nhiệm vụ phân loại.

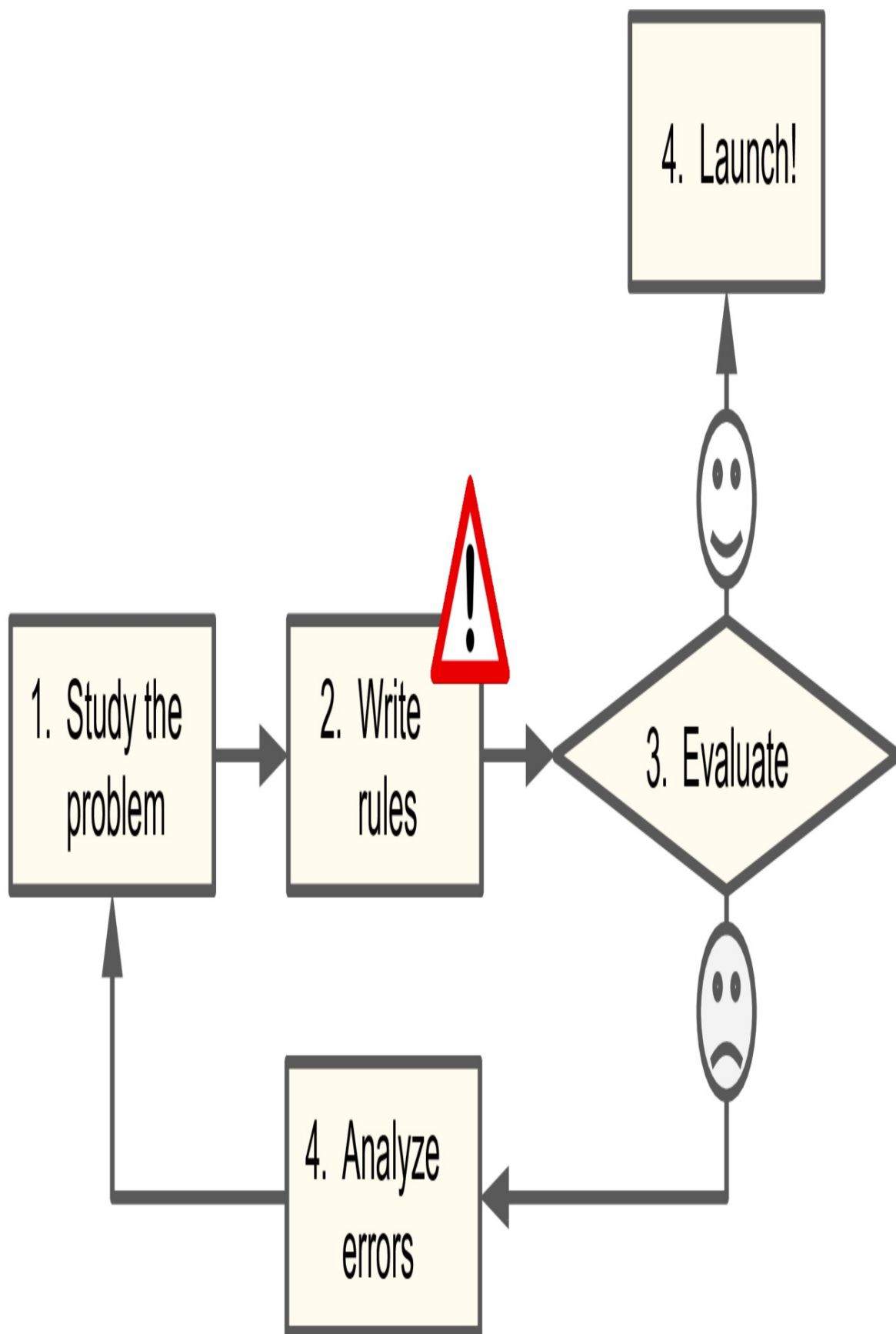
Nếu bạn chỉ tải xuống một bản sao của tất cả các bài viết trên Wikipedia, máy tính của bạn sẽ có nhiều dữ liệu hơn, nhưng nó không đột nhiên trở nên giỏi hơn trong bất kỳ tác vụ nào. Đó không phải là Học máy.

Tại sao nên sử dụng máy học?

Hãy xem xét cách bạn sẽ viết một bộ lọc thư rác bằng các kỹ thuật lập trình truyền thống (Hình 1-1):

1. Đầu tiên, bạn cần xem thư rác thường trông như thế nào. Bạn có thể...
Bạn sẽ nhận thấy một số từ hoặc cụm từ (như "4U," "thẻ tín dụng," "miễn phí," và "tuyệt vời") thường xuất hiện rất nhiều trong tiêu đề email. Có lẽ bạn cũng sẽ nhận thấy một vài mẫu tương tự khác trong tên người gửi, nội dung email và các phần khác của email.

2. Bạn sẽ viết một thuật toán phát hiện cho mỗi mẫu mà bạn nhận thấy, và chương trình của bạn sẽ gắn cờ các email là thư rác nếu phát hiện thấy một số lượng nhất định các mẫu này.
3. Bạn sẽ kiểm tra chương trình của mình và lặp lại bước 1 và 2 cho đến khi nó đủ tốt để triển khai.



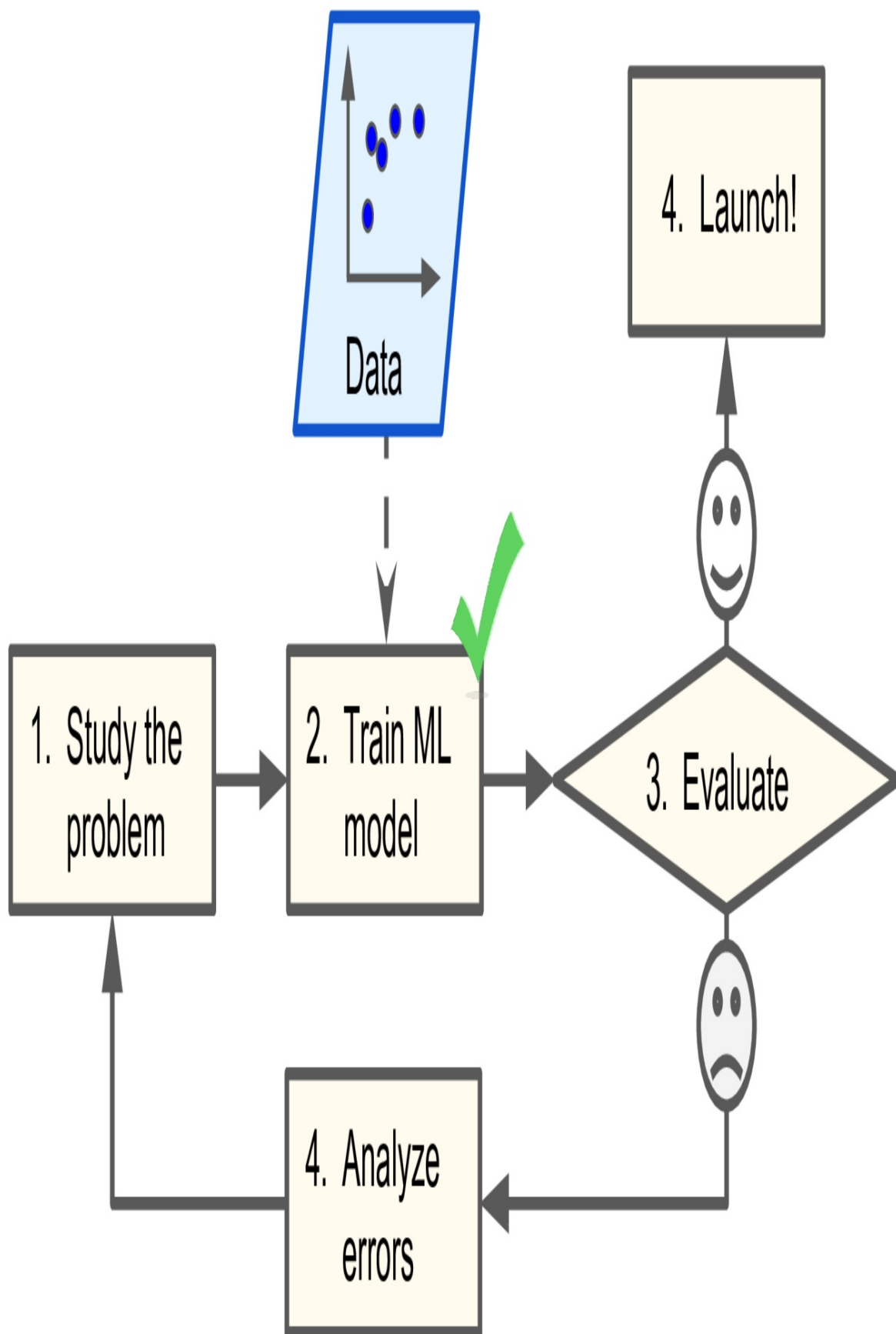
Hình 1-1. Phương pháp truyền thống

Vì bài toán khó, chương trình của bạn có thể sẽ trở thành một danh sách dài các quy tắc phức tạp – rất khó bảo trì.

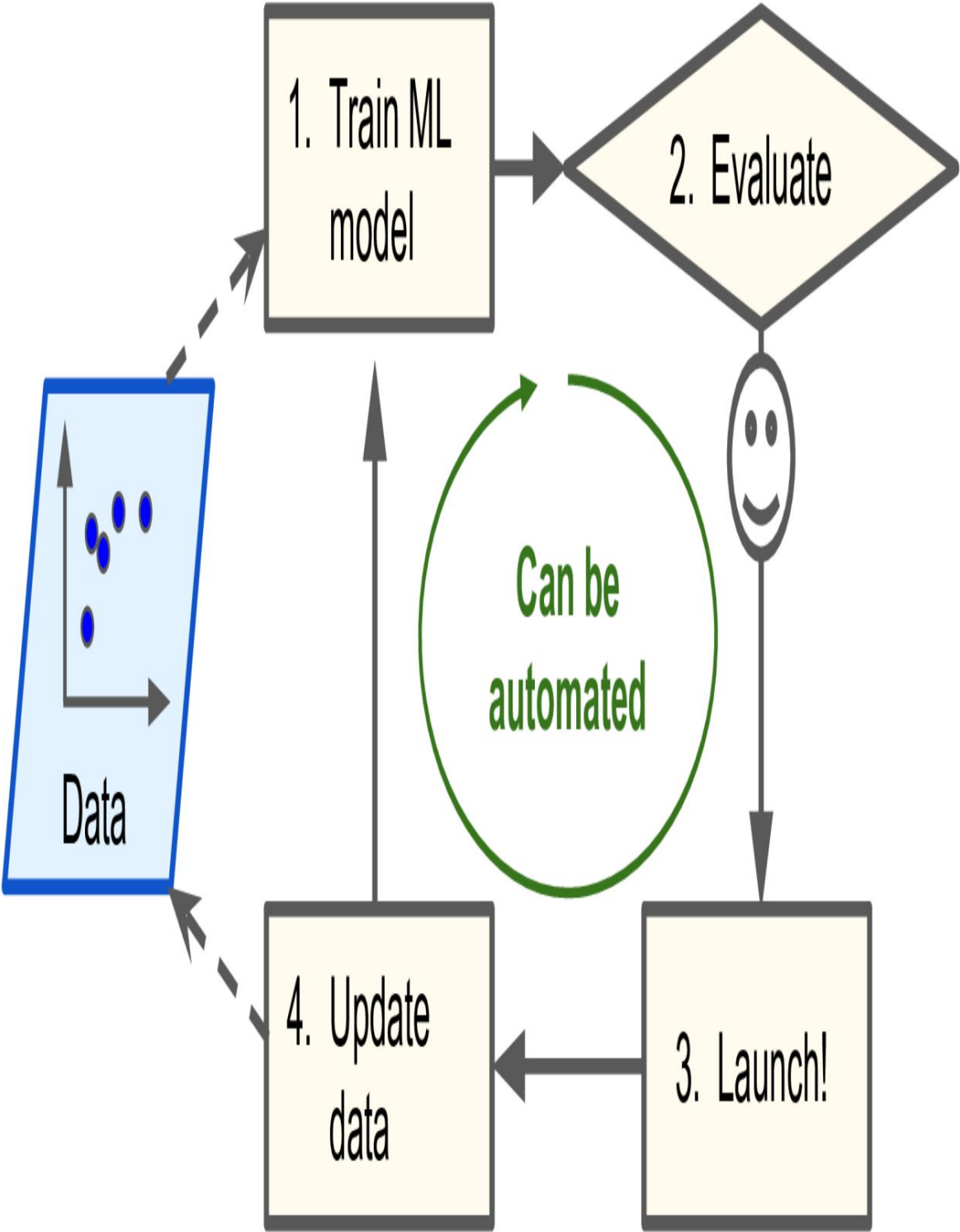
Ngược lại, bộ lọc thư rác dựa trên các kỹ thuật Học máy tự động học cách nhận biết những từ và cụm từ nào là dấu hiệu dự đoán thư rác tốt bằng cách phát hiện các mẫu từ xuất hiện thường xuyên bất thường trong các ví dụ thư rác so với các ví dụ thư hợp lệ (Hình 1-2). Chương trình này ngắn gọn hơn, dễ bảo trì hơn và có khả năng chính xác hơn.

Điều gì sẽ xảy ra nếu những kẻ gửi thư rác nhận thấy tất cả email của chúng có chứa "4U" đều bị chặn? Họ có thể bắt đầu viết "Dành cho bạn" thay vì "Dành cho bạn". Bộ lọc thư rác sử dụng các kỹ thuật lập trình truyền thống cần được cập nhật để gắn cờ các email "Dành cho bạn". Nếu những kẻ gửi thư rác liên tục tìm cách vượt qua bộ lọc thư rác của bạn, bạn sẽ cần phải liên tục viết ra các quy tắc mới.

Ngược lại, bộ lọc thư rác dựa trên các kỹ thuật Học máy sẽ tự động nhận thấy rằng cụm từ "For U" xuất hiện với tần suất bất thường trong thư rác do người dùng báo cáo và bắt đầu gắn cờ chúng mà không cần sự can thiệp của bạn (Hình 1-3).



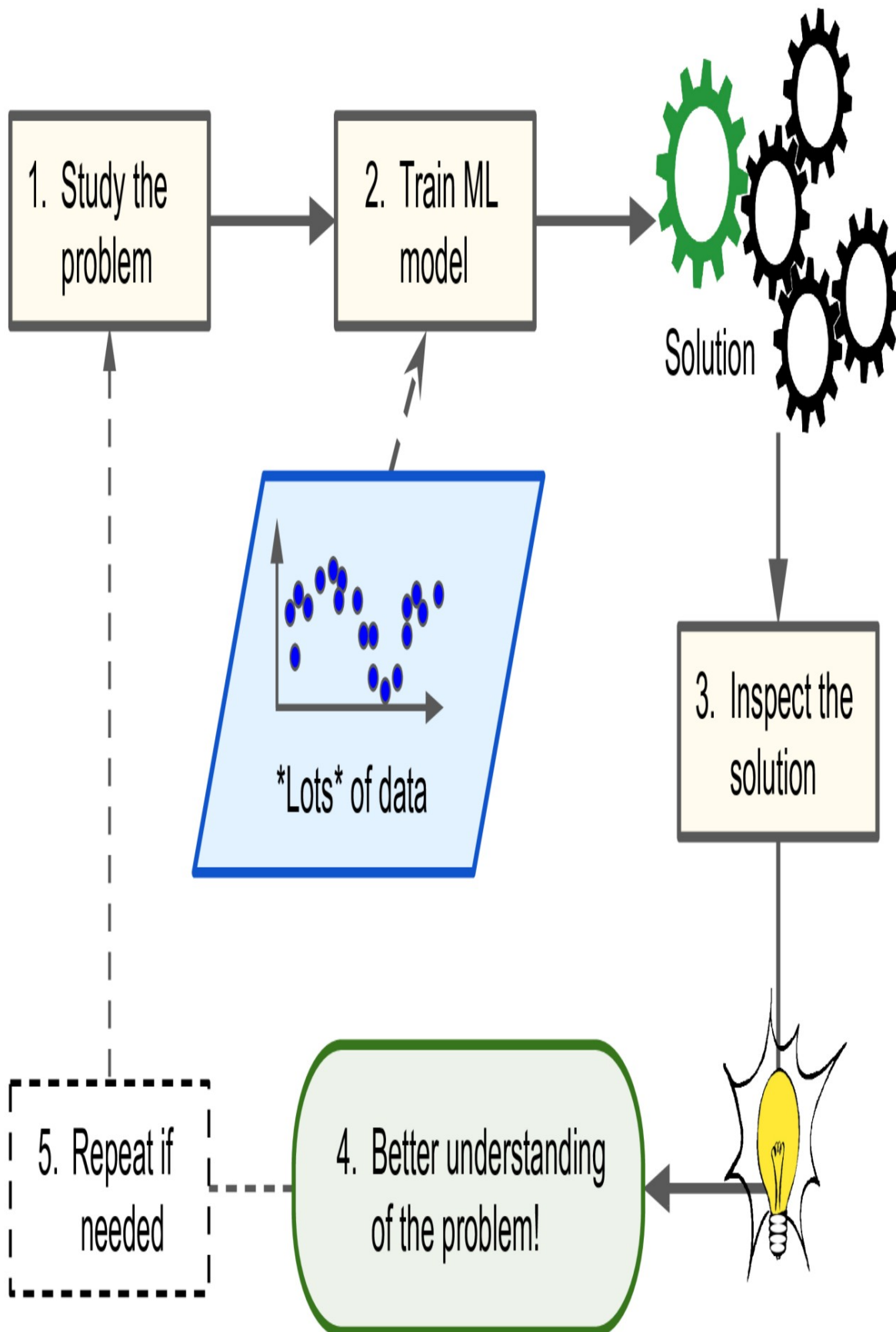
Hình 1-2. Phương pháp học máy



Hình 1-3. Tự động thích ứng với sự thay đổi

Một lĩnh vực khác mà Học máy tỏa sáng là giải quyết các vấn đề quá phức tạp đối với các phương pháp truyền thống hoặc không có thuật toán nào được biết đến. Ví dụ, hãy xem xét nhận dạng giọng nói. Giả sử bạn muốn bắt đầu đơn giản và viết một chương trình có khả năng phân biệt các từ “một” và “hai”. Bạn có thể nhận thấy rằng từ “hai” bắt đầu bằng một âm cao (“T”), vì vậy bạn có thể lập trình sẵn một thuật toán đo cường độ âm cao và sử dụng nó để phân biệt một và hai – nhưng rõ ràng kỹ thuật này sẽ không thể mở rộng quy mô cho hàng nghìn từ được nói bởi hàng triệu người rất khác nhau trong môi trường ồn ào và bằng hàng chục ngôn ngữ. Giải pháp tốt nhất (ít nhất là hiện nay) là viết một thuật toán tự học, dựa trên nhiều bản ghi âm ví dụ cho mỗi từ.

Cuối cùng, Học máy có thể giúp con người học hỏi (Hình 1-4). Các mô hình Học máy có thể được kiểm tra để xem chúng đã học được gì (mặc dù đối với một số mô hình thì điều này có thể khá phức tạp). Ví dụ, sau khi một bộ lọc thư rác đã được huấn luyện với đủ lượng thư rác, nó có thể dễ dàng được kiểm tra để tìm ra danh sách các từ và sự kết hợp các từ mà nó cho là những yếu tố dự đoán thư rác tốt nhất. Đôi khi điều này sẽ hé lộ những mối tương quan không ngờ tới hoặc xu hướng mới, từ đó dẫn đến sự hiểu biết tốt hơn về vấn đề. Việc phân tích lượng lớn dữ liệu để khám phá các mô hình ẩn được gọi là khai thác dữ liệu, và Học máy (Machine Learning) rất giỏi trong lĩnh vực này.



Hình 1-4. Máy học có thể giúp con người học hỏi.

Tóm lại, Học máy rất hữu ích cho:

- Những vấn đề mà các giải pháp hiện có đòi hỏi nhiều tình hình hoặc danh sách dài các quy tắc: một mô hình Học máy thường có thể đơn giản hóa mã và hoạt động tốt hơn phương pháp truyền thống.
- Những vấn đề phức tạp mà phương pháp truyền thống không mang lại giải pháp hiệu quả: có lẽ chỉ những kỹ thuật Học máy tiên tiến nhất mới có thể tìm ra lời giải.
- Môi trường biến động: Hệ thống Học máy có thể dễ dàng được huấn luyện lại trên dữ liệu mới, luôn luôn cập nhật.
- Thu thập thông tin chi tiết về các vấn đề phức tạp và lượng dữ liệu lớn.

Ví dụ về các ứng dụng

Hãy cùng xem xét một số ví dụ cụ thể về các nhiệm vụ Học máy, cùng với các kỹ thuật có thể giải quyết chúng:

Phân tích hình ảnh sản phẩm trên dây chuyền sản xuất để tự động phân loại chúng.

Đây là phân loại hình ảnh, thường được thực hiện bằng cách sử dụng mạng nơ-ron tích chập (CNN; xem [Chương 14](#)) hoặc đôi khi là Transformer (xem [Chương 16](#)).

Phát hiện khối u trong ảnh chụp não

Đây là phân đoạn ảnh ngữ nghĩa, trong đó mỗi pixel trong ảnh được phân loại (vì chúng ta muốn xác định vị trí và hình dạng chính xác của khối u), thường sử dụng mạng nơ-ron tích chập (CNN) hoặc Transformer.

Tự động phân loại bài báo

Đây là xử lý ngôn ngữ tự nhiên (NLP), và cụ thể hơn là phân loại văn bản, có thể được giải quyết bằng cách sử dụng mạng nơ-ron hồi quy.

(RNN) và CNN, nhưng Transformer hoạt động thậm chí còn tốt hơn (xem [Chương 16](#)).

Tự động gắn cờ các bình luận xúc phạm trên diễn đàn thảo luận

Đây cũng là phân loại văn bản, sử dụng cùng các công cụ xử lý ngôn ngữ tự nhiên (NLP).

Tự động tóm tắt các tài liệu dài.

Đây là một nhánh của xử lý ngôn ngữ tự nhiên (NLP) được gọi là tóm tắt văn bản, cũng sử dụng các công cụ tương tự.

Tạo chatbot hoặc trợ lý cá nhân

Điều này bao gồm nhiều thành phần xử lý ngôn ngữ tự nhiên (NLP), trong đó có các mô-đun hiểu ngôn ngữ tự nhiên (NLU) và trả lời câu hỏi.

Dự báo doanh thu của công ty bạn trong năm tới, dựa trên nhiều chỉ số hiệu suất.

Đây là một bài toán hồi quy (tức là dự đoán giá trị) có thể được giải quyết bằng bất kỳ mô hình hồi quy nào, chẳng hạn như mô hình hồi quy tuyến tính hoặc hồi quy đa thức (xem [Chương 4](#)), SVM hồi quy (xem [Chương 5](#)), rừng ngẫu nhiên hồi quy (xem [Chương 7](#)) hoặc mạng nơ-ron nhân tạo (xem [Chương 10](#)). Nếu bạn muốn tính đến chuỗi các chỉ số hiệu suất trong quá khứ, bạn có thể muốn sử dụng RNN, CNN hoặc Transformer (xem [Chương 15](#) và [16](#)).

Giúp ứng dụng của bạn phản hồi lệnh thoại

Đây là công nghệ nhận dạng giọng nói, đòi hỏi xử lý các mẫu âm thanh: vì chúng là các chuỗi dài và phức tạp, nên chúng thường được xử lý bằng RNN, CNN hoặc Transformer (xem [Chương 15](#) và [16](#)).

Phát hiện gian lận thẻ tín dụng

Đây là bài toán phát hiện bất thường, có thể được giải quyết bằng cách sử dụng rừng cô lập, mô hình hỗn hợp Gaussian (xem [Chương 9](#)) hoặc bộ mã hóa tự động (xem [Chương 17](#)).

Phân khúc khách hàng dựa trên hành vi mua hàng của họ để bạn có thể thiết kế chiến lược tiếp thị khác nhau cho từng phân khúc.

Đây là quá trình phân cụm, có thể được thực hiện bằng cách sử dụng K-Means, DBSCAN và nhiều thuật toán khác (xem [Chương 9](#)).

Biểu diễn một tập dữ liệu phức tạp, đa chiều thành một sơ đồ rõ ràng và dễ hiểu.

Đây là hình thức trực quan hóa dữ liệu, thường liên quan đến các kỹ thuật giảm chiều dữ liệu (xem [Chương 8](#)).

Đề xuất sản phẩm mà khách hàng có thể quan tâm, dựa trên lịch sử mua hàng trước đó.

Đây là một hệ thống đề xuất. Một cách tiếp cận là đưa dữ liệu về các giao dịch mua hàng trước đây (và các thông tin khác về khách hàng) vào mạng nơ-ron nhân tạo (xem [Chương 10](#)), và yêu cầu nó đưa ra đề xuất về giao dịch mua hàng tiếp theo có khả năng xảy ra nhất. Mạng nơ-ron này thường được huấn luyện dựa trên chuỗi giao dịch mua hàng trước đây của tất cả khách hàng.

Xây dựng một bot thông minh cho trò chơi

Vấn đề này thường được giải quyết bằng cách sử dụng Học tăng cường (Reinforcement Learning - RL; xem [Chương 18](#)), một nhánh của Học máy (Machine Learning) huấn luyện các tác nhân (như bot) để chọn các hành động nhằm tối đa hóa phần thưởng của chúng theo thời gian (ví dụ: bot có thể nhận được phần thưởng mỗi khi người chơi mất điểm sinh mệnh), trong một môi trường nhất định (như trò chơi). Chương trình AlphaGo nổi tiếng đã đánh bại nhà vô địch thế giới cờ vây được xây dựng bằng RL.

Danh sách này có thể kéo dài mãi, nhưng hy vọng nó sẽ giúp bạn hình dung được phạm vi và độ phức tạp đáng kinh ngạc của các nhiệm vụ mà Học máy có thể giải quyết, cũng như các loại kỹ thuật bạn sẽ sử dụng cho mỗi nhiệm vụ.

Các loại hệ thống học máy

Có rất nhiều loại hệ thống Học máy khác nhau, vì vậy việc phân loại chúng thành các nhóm lớn dựa trên các tiêu chí sau đây là rất hữu ích:

- Cách thức giám sát trong quá trình đào tạo (có giám sát, không giám sát, giám sát một phần, tự giám sát và các hình thức khác)
- Liệu họ có thể học từng bước một cách nhanh chóng (học trực tuyến so với học theo nhóm) hay không.
- Cho dù chúng hoạt động bằng cách đơn giản so sánh các điểm dữ liệu mới với các điểm dữ liệu đã biết, hay bằng cách phát hiện các mẫu trong dữ liệu huấn luyện và xây dựng mô hình dự đoán, tương tự như cách các nhà khoa học làm (học dựa trên trường hợp so với học dựa trên mô hình).

Các tiêu chí này không loại trừ lẫn nhau; bạn có thể kết hợp chúng theo bất kỳ cách nào bạn muốn. Ví dụ, một bộ lọc thư rác hiện đại có thể học hỏi tức thời bằng cách sử dụng mô hình mạng nơ-ron sâu được huấn luyện bằng các ví dụ về thư rác và thư hợp lệ do con người cung cấp; điều này biến nó thành một hệ thống học tập có giám sát, dựa trên mô hình và trực tuyến.

Chúng ta hãy cùng xem xét kỹ hơn từng tiêu chí này.

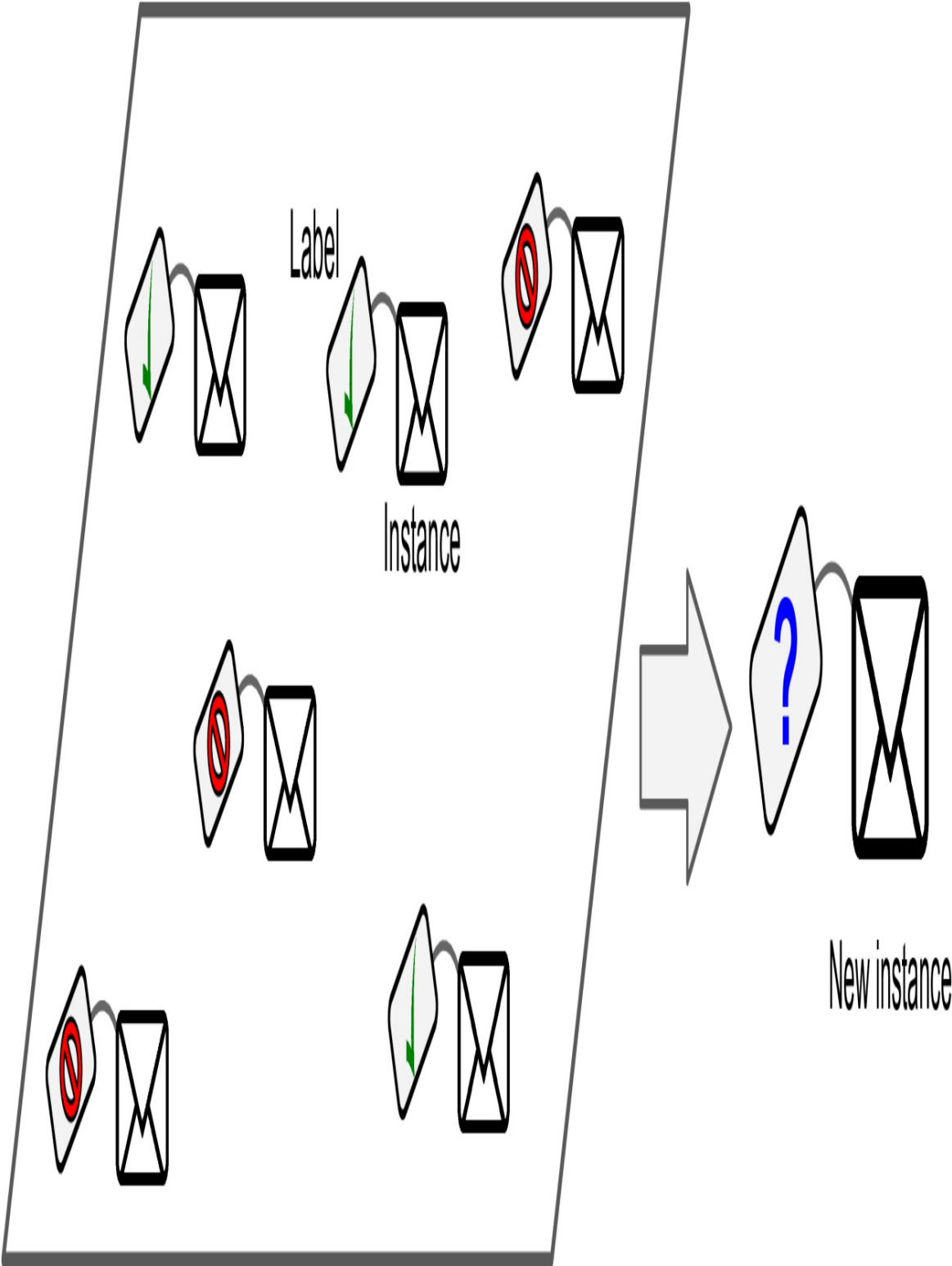
Giám sát đào tạo

Các hệ thống học máy có thể được phân loại dựa trên lượng và loại hình giám sát mà chúng nhận được trong quá trình huấn luyện. Có nhiều loại, nhưng chúng ta sẽ thảo luận về những loại chính: học có giám sát, học không giám sát, học tự giám sát, học bán giám sát và học tăng cường.

Học có giám sát Trong học có

giám sát, tập dữ liệu huấn luyện mà bạn cung cấp cho thuật toán bao gồm các giải pháp mong muốn, được gọi là nhãn (Hình 1-5).

Training set



Hình 1-5. Tập dữ liệu huấn luyện có nhãn để phân loại thư rác (một ví dụ về học có giám sát)

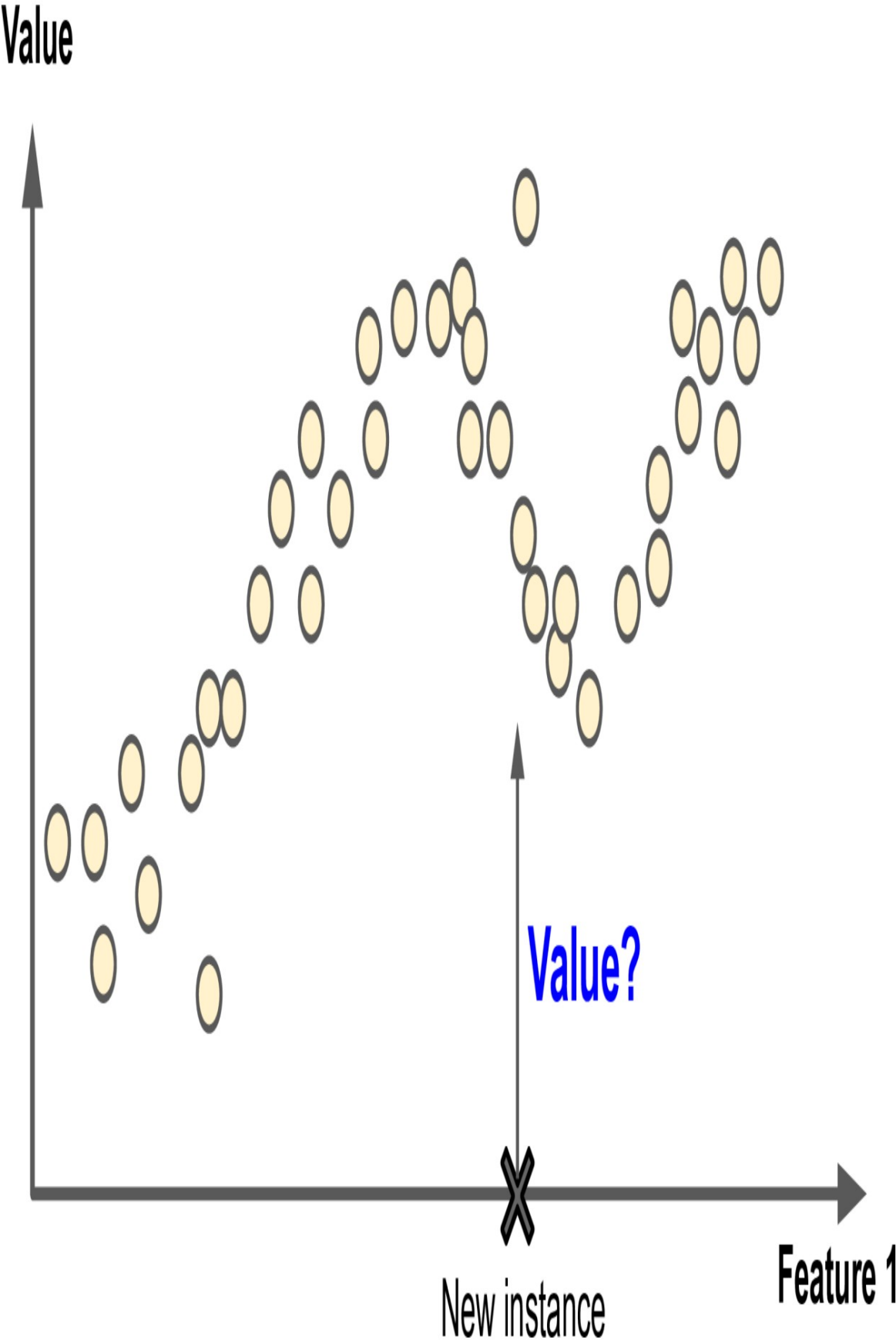
Phân loại là một nhiệm vụ học có giám sát điển hình. Bộ lọc thư rác là một ví dụ điển hình: nó được huấn luyện với nhiều email ví dụ cùng với loại thư của chúng (thư rác hoặc thư hợp lệ), và nó phải học cách phân loại các email mới.

Một nhiệm vụ điển hình khác là dự đoán một giá trị số mục tiêu, chẳng hạn như giá của một chiếc ô tô, dựa trên một tập hợp các đặc điểm (số km đã đi, tuổi đời, thương hiệu, v.v.). Loại nhiệm vụ này được gọi là hồi quy (Hình 1-6). Để huấn luyện hệ thống, bạn cần cung cấp cho nó nhiều ví dụ về ô tô, bao gồm cả các đặc điểm và mục tiêu của chúng (tức là giá cả).

GHI CHÚ

Trong học có giám sát, các từ "mục tiêu" và "nhãn" thường được coi là đồng nghĩa, nhưng "mục tiêu" phổ biến hơn trong các bài toán hồi quy, còn "nhãn" phổ biến hơn trong các bài toán phân loại. Hơn nữa, các đặc trưng đôi khi được gọi là các biến dự đoán hoặc thuộc tính. Các thuật ngữ này có thể đề cập đến các mẫu riêng lẻ (ví dụ: "đặc trưng quãng đường đi được của chiếc xe này bằng 15.000") hoặc đến tất cả các mẫu (ví dụ: "đặc trưng quãng đường đi được có mối tương quan mạnh với giá cả").

Lưu ý rằng một số mô hình hồi quy cũng có thể được sử dụng cho phân loại, và ngược lại. Ví dụ, hồi quy logistic thường được sử dụng cho phân loại, vì nó có thể đưa ra giá trị tương ứng với xác suất thuộc về một lớp nhất định (ví dụ: 20% khả năng là thư rác).

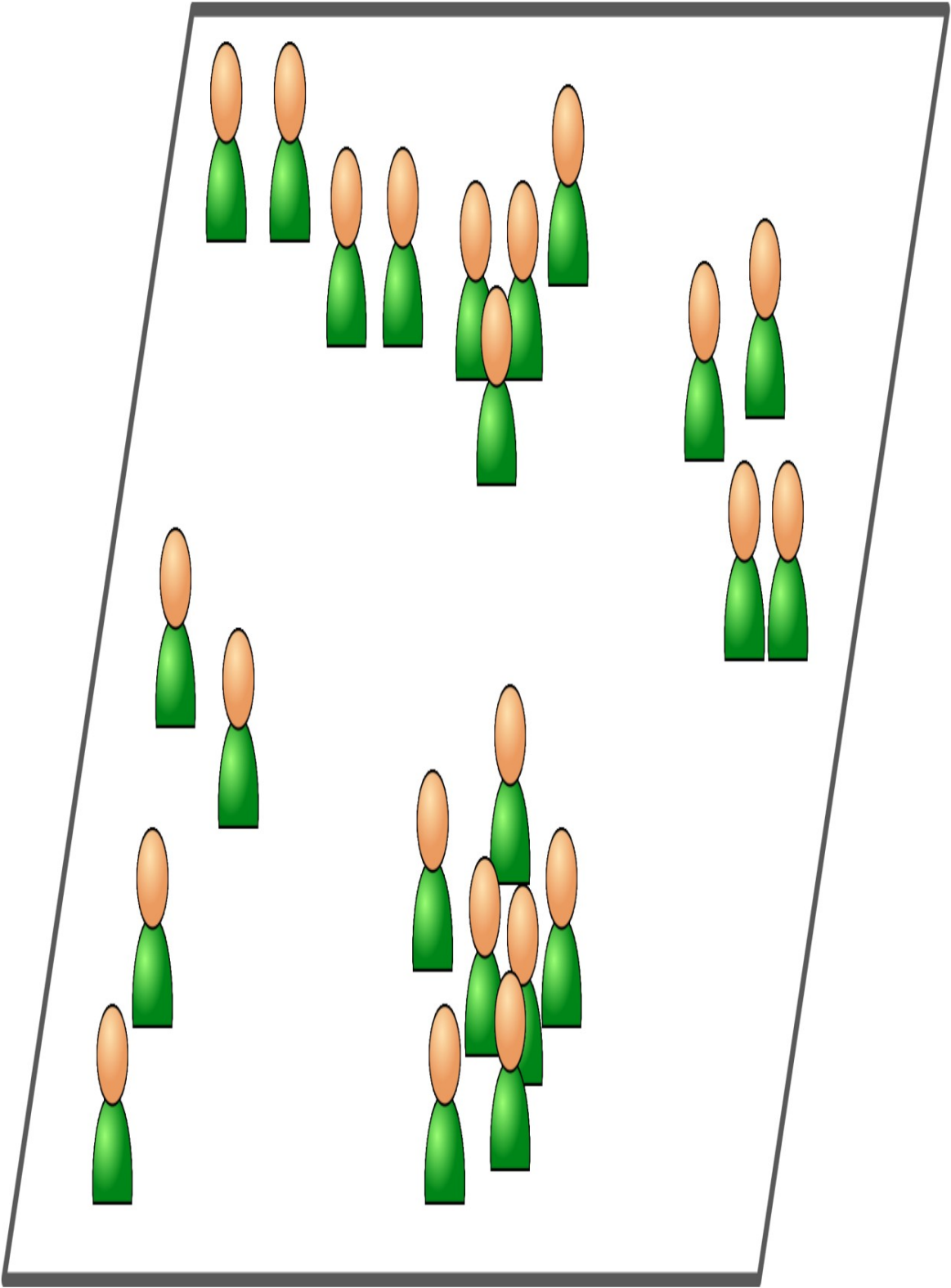


Hình 1-6. Bài toán hồi quy: dự đoán một giá trị, dựa trên một đặc trưng đầu vào (thường có nhiều đặc trưng đầu vào, và đôi khi có nhiều giá trị đầu ra)

Học không giám sát

Trong học không giám sát, như bạn có thể đoán, dữ liệu huấn luyện không được gắn nhãn (Hình 1-7). Hệ thống cố gắng học mà không cần người hướng dẫn.

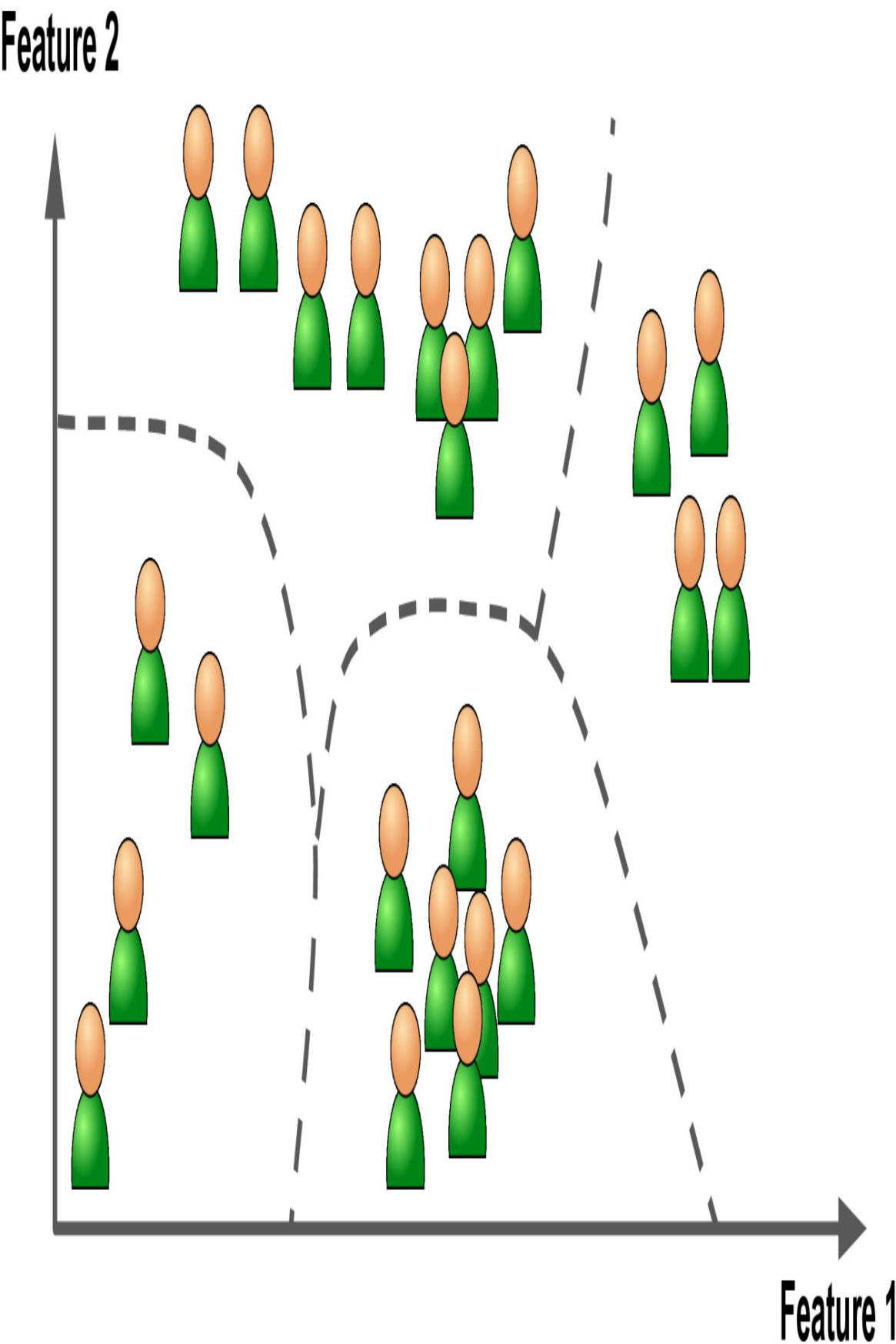
Training set



Hình 1-7. Tập dữ liệu huấn luyện không nhãn cho học không giám sát.

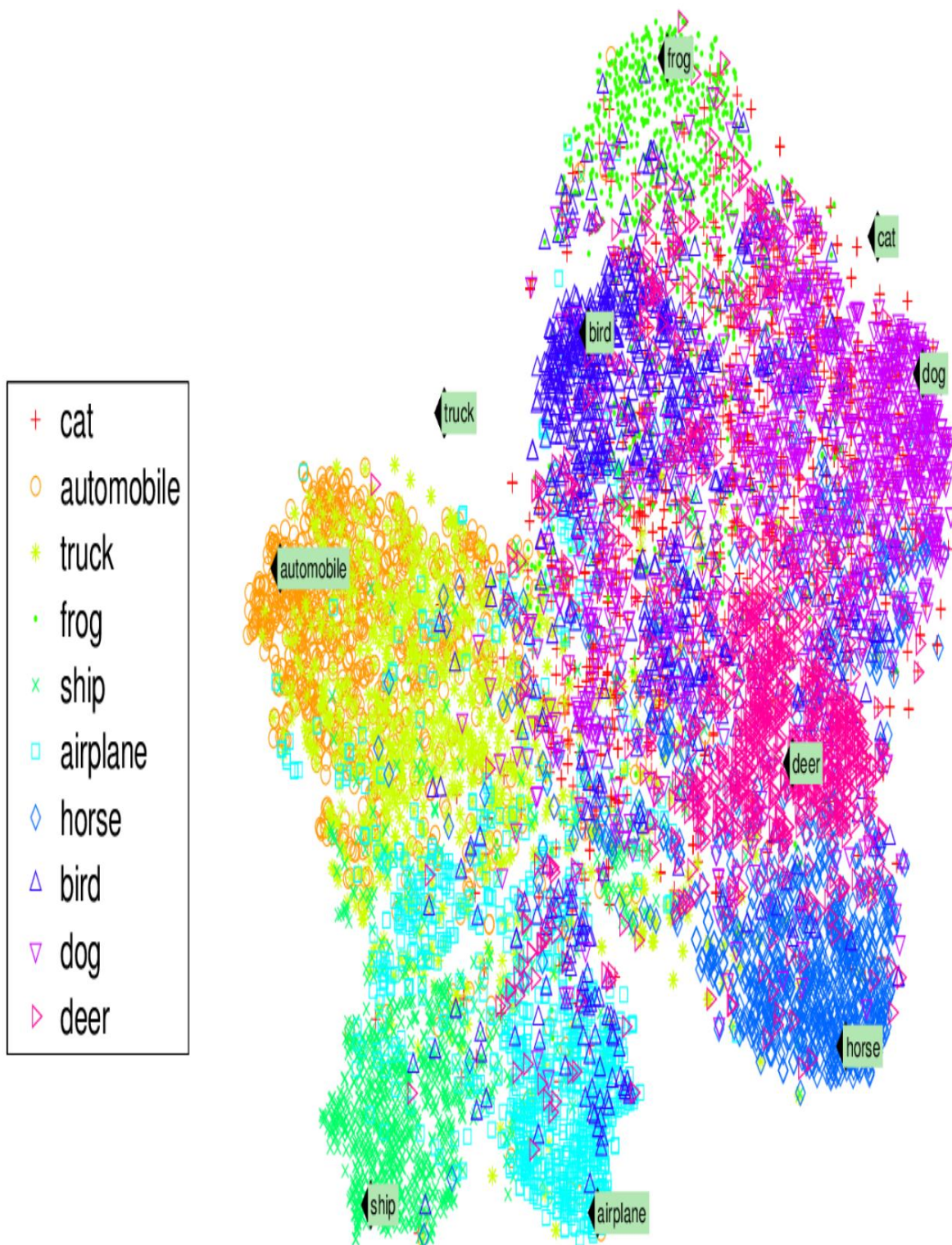
Ví dụ, giả sử bạn có rất nhiều dữ liệu về khách truy cập blog của mình. Bạn có thể muốn chạy thuật toán phân cụm để cố gắng phát hiện các nhóm khách truy cập tương tự nhau (Hình 1-8). Bạn không cần phải cho thuật toán biết khách truy cập thuộc nhóm nào: nó sẽ tự tìm ra các mối liên hệ đó mà không cần sự trợ giúp của bạn. Ví dụ, nó có thể nhận thấy rằng 40% khách truy cập của bạn là thanh thiếu niên yêu thích truyện tranh và thường đọc blog của bạn sau giờ học, trong khi 20% là người lớn thích khoa học viễn tưởng và truy cập vào cuối tuần. Nếu bạn sử dụng thuật toán phân cụm phân cấp, nó cũng có thể chia mỗi nhóm thành các nhóm nhỏ hơn.

Điều này có thể giúp bạn nhắm mục tiêu bài đăng của mình cho từng nhóm đối tượng.



Hình 1-8. Phân cụm

Các thuật toán trực quan hóa cũng là những ví dụ điển hình về học không giám sát: bạn cung cấp cho chúng rất nhiều dữ liệu phức tạp và chưa được gắn nhãn, và chúng sẽ xuất ra một biểu diễn 2D hoặc 3D của dữ liệu mà bạn có thể dễ dàng vẽ đồ thị (Hình 1-9). Các thuật toán này cố gắng bảo toàn cấu trúc càng nhiều càng tốt (ví dụ: cố gắng giữ cho các cụm riêng biệt trong không gian đầu vào không chồng chéo lên nhau trong hình ảnh trực quan) để bạn có thể hiểu cách dữ liệu được tổ chức và có thể xác định các mẫu không ngờ tới.



Hình 1-9. Ví dụ về hình ảnh trực quan t-SNE làm nổi bật các cụm ngữ nghĩa.²

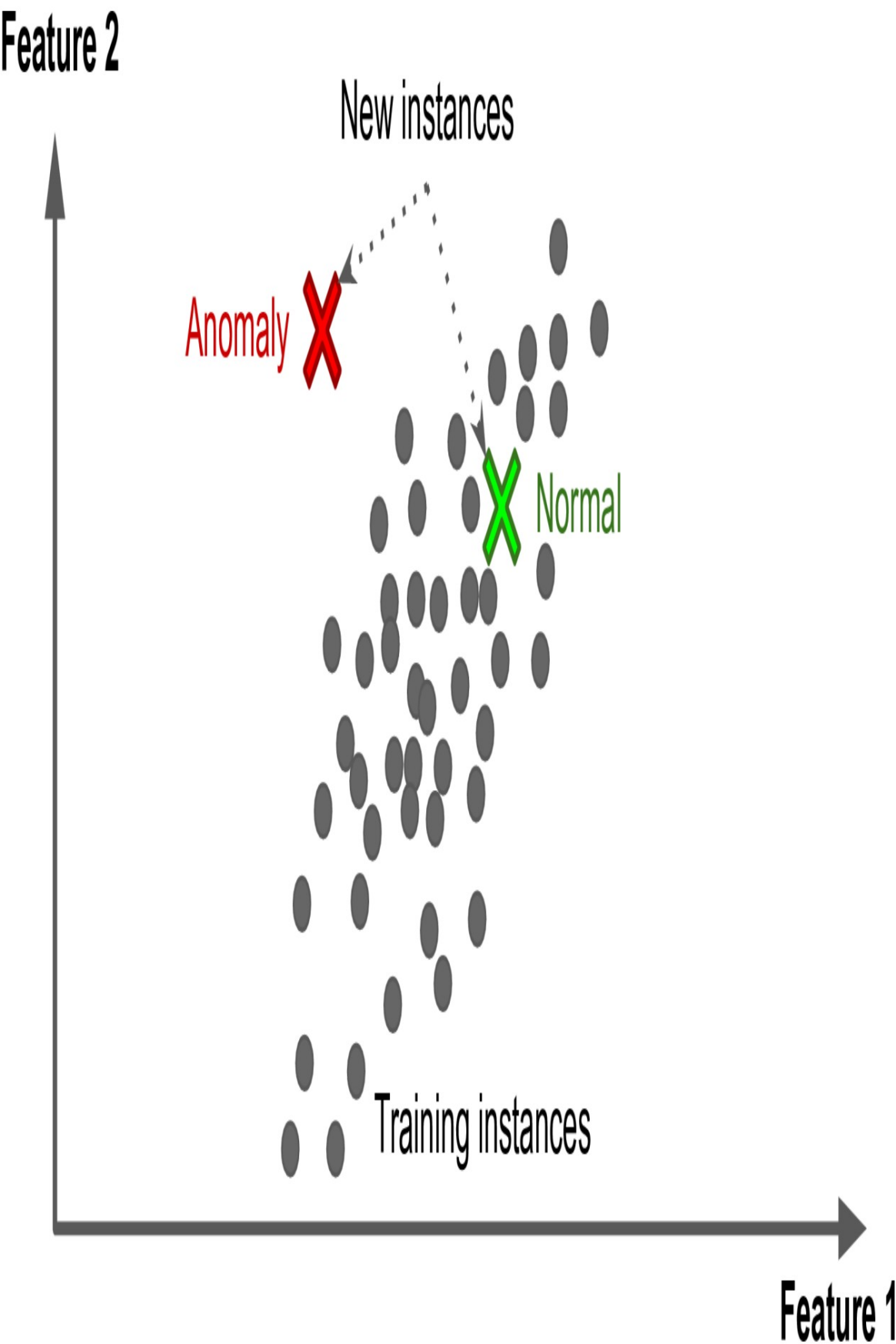
Một nhiệm vụ liên quan là giảm chiều dữ liệu, trong đó mục tiêu là đơn giản hóa dữ liệu mà không làm mất quá nhiều thông tin. Một cách để làm điều này là...

Thuật toán giảm chiều dữ liệu kết hợp nhiều đặc điểm có tương quan với nhau thành một đặc điểm duy nhất. Ví dụ, số km đã đi của một chiếc xe có thể tương quan chặt chẽ với tuổi đời của nó, vì vậy thuật toán giảm chiều dữ liệu sẽ kết hợp chúng thành một đặc điểm duy nhất thể hiện mức độ hao mòn của xe. Quá trình này được gọi là trích xuất đặc điểm.

MẸO

Thường thì việc giảm kích thước dữ liệu huấn luyện bằng thuật toán giảm chiều trước khi đưa vào một thuật toán Học máy khác (chẳng hạn như thuật toán học có giám sát) là một ý tưởng hay. Quá trình này sẽ chạy nhanh hơn, dữ liệu sẽ chiếm ít dung lượng ổ đĩa và bộ nhớ hơn, và trong một số trường hợp, hiệu suất cũng có thể tốt hơn.

Một nhiệm vụ quan trọng khác không cần giám sát là phát hiện bất thường—ví dụ, phát hiện các giao dịch thẻ tín dụng bất thường để ngăn chặn gian lận, phát hiện lỗi sản xuất hoặc tự động loại bỏ các giá trị ngoại lai khỏi tập dữ liệu trước khi đưa vào thuật toán học khác. Hệ thống được cho xem chủ yếu các trường hợp bình thường trong quá trình huấn luyện, vì vậy nó học cách nhận biết chúng; sau đó, khi nó thấy một trường hợp mới, nó có thể cho biết liệu trường hợp đó trông giống như một trường hợp bình thường hay có khả năng là một bất thường (xem [Hình 1-10](#)). Một nhiệm vụ rất tương tự là phát hiện sự mới lạ: nó nhằm mục đích phát hiện các trường hợp mới trông khác biệt so với tất cả các trường hợp trong tập huấn luyện. Điều này yêu cầu phải có một tập huấn luyện rất "sạch", không có bất kỳ trường hợp nào mà bạn muốn thuật toán phát hiện. Ví dụ, nếu bạn có hàng nghìn bức ảnh về chó, và 1% trong số những bức ảnh này là chó Chihuahua, thì thuật toán phát hiện sự mới lạ không nên coi những bức ảnh mới về chó Chihuahua là mới lạ. Mặt khác, các thuật toán phát hiện bất thường có thể coi những con chó này là rất hiếm và rất khác biệt so với những con chó khác đến mức chúng có thể phân loại chúng là bất thường (không có ý xúc phạm chó Chihuahua).



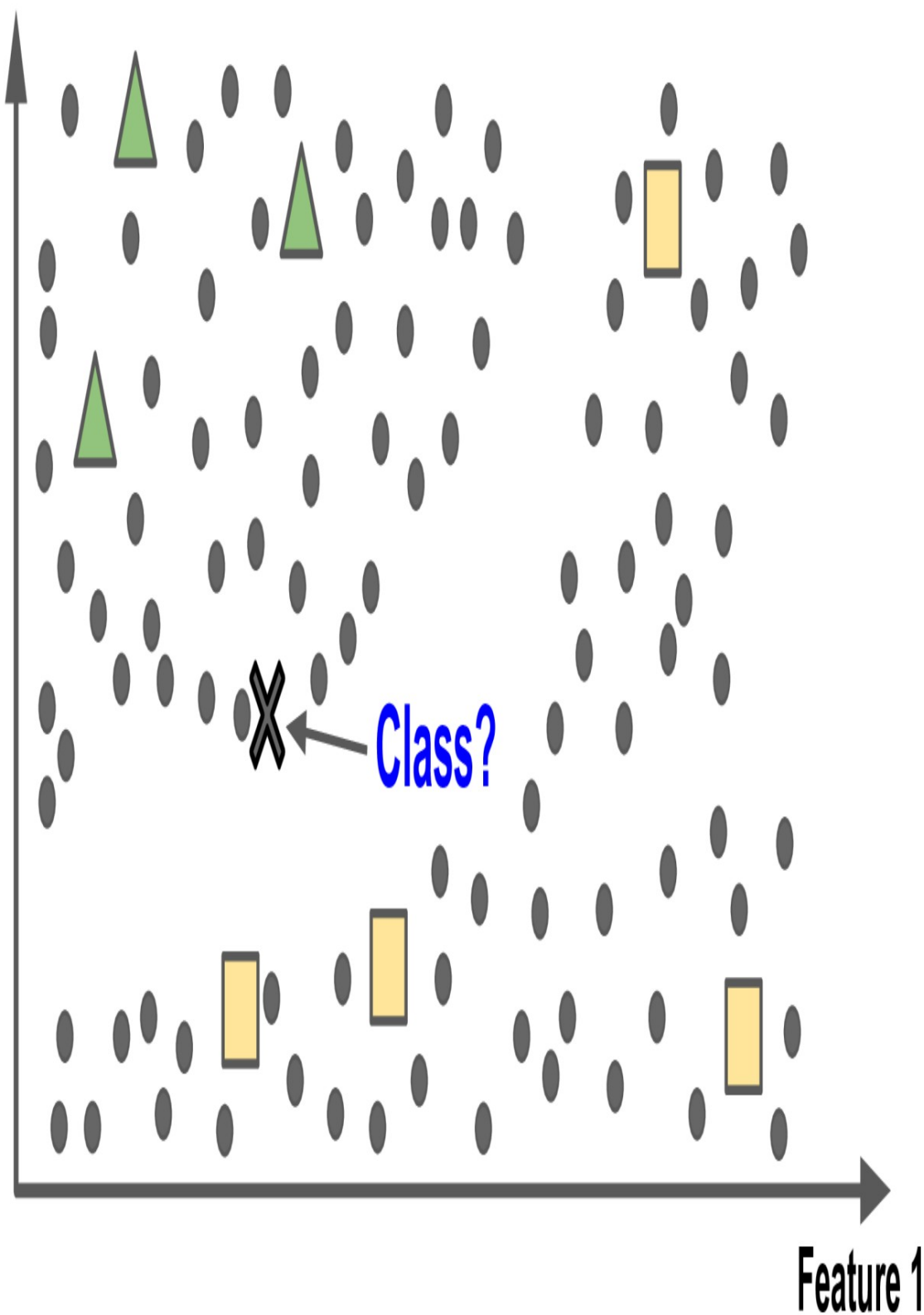
Hình 1-10. Phát hiện bất thường

Cuối cùng, một nhiệm vụ học không giám sát phổ biến khác là học luật liên kết, trong đó mục tiêu là phân tích lượng lớn dữ liệu và khám phá các mối quan hệ thú vị giữa các thuộc tính. Ví dụ, giả sử bạn sở hữu một siêu thị. Áp dụng quy tắc liên kết trên nhật ký bán hàng của bạn có thể cho thấy rằng những người mua tương ớt và khoai tây chiên cũng có xu hướng mua bí tết. Do đó, bạn có thể muốn đặt những mặt hàng này gần nhau.

Học bán giám sát

Vì việc gắn nhãn dữ liệu thường tốn thời gian và chi phí, bạn thường sẽ có rất nhiều trường hợp chưa được gắn nhãn và rất ít trường hợp đã được gắn nhãn. Một số thuật toán có thể xử lý dữ liệu được gắn nhãn một phần. Điều này được gọi là học bán giám sát (Hình 1-11).

Feature 2



Hình 1-11. Học bán giám sát với hai lớp (tam giác và hình vuông): các ví dụ chưa được gắn nhãn (hình tròn) giúp phân loại một trường hợp mới (hình chữ thập) vào lớp tam giác thay vì lớp hình vuông, mặc dù nó gần với các hình vuông đã được gắn nhãn hơn.

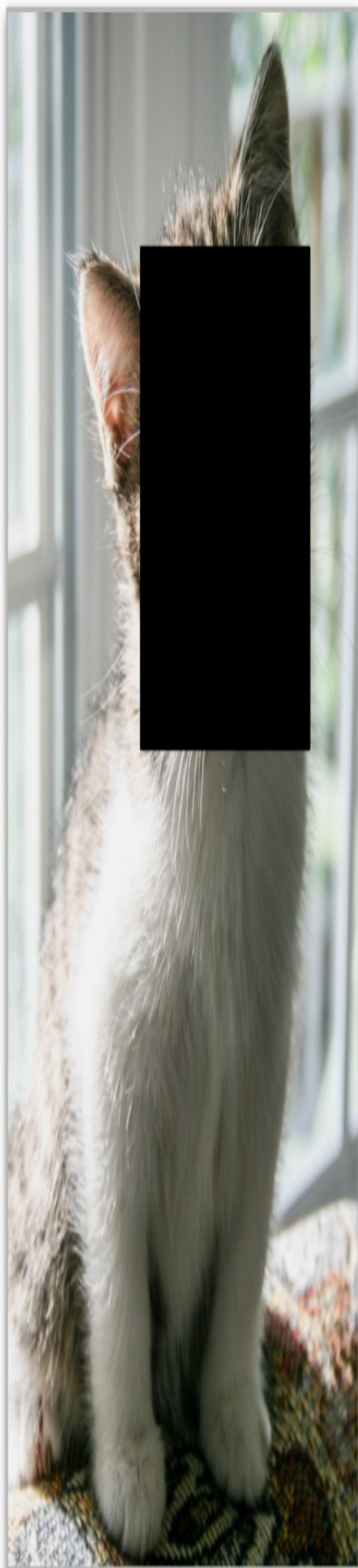
Một số dịch vụ lưu trữ ảnh, chẳng hạn như Google Photos, là những ví dụ điển hình. Sau khi bạn tải tất cả ảnh gia đình lên dịch vụ, hệ thống sẽ tự động nhận ra rằng cùng một người A xuất hiện trong ảnh số 1, 5 và 11, trong khi một người khác B xuất hiện trong ảnh số 2, 5 và 7. Đây là phần không giám sát của thuật toán (phân cụm). Bây giờ, tất cả những gì hệ thống cần là bạn cho nó biết những người này là ai. Chỉ cần thêm một nhãn cho mỗi người và hệ thống có thể đặt tên cho mọi người trong mọi bức ảnh, điều này ³ rất hữu ích cho việc tìm kiếm ảnh.

Hầu hết các thuật toán học bán giám sát là sự kết hợp giữa các thuật toán học không giám sát và học giám sát. Ví dụ, một thuật toán phân cụm có thể được sử dụng để nhóm các trường hợp tương tự lại với nhau, và sau đó mỗi trường hợp chưa được gắn nhãn có thể được gắn nhãn phổ biến nhất trong cụm của chúng. Sau khi toàn bộ tập dữ liệu được gắn nhãn, có thể sử dụng bất kỳ thuật toán học giám sát nào.

Học tập tự giám sát

Một cách tiếp cận khác trong Học máy là tạo ra một tập dữ liệu được gán nhãn đầy đủ từ một tập dữ liệu hoàn toàn chưa được gán nhãn. Một lần nữa, khi toàn bộ tập dữ liệu đã được gán nhãn, bất kỳ thuật toán học có giám sát nào cũng có thể được sử dụng. Cách tiếp cận này được gọi là học tự giám sát.

Ví dụ, nếu bạn có một tập dữ liệu lớn gồm các hình ảnh chưa được gắn nhãn, bạn có thể ngẫu nhiên che một phần nhỏ của mỗi hình ảnh và sau đó huấn luyện một mô hình để khôi phục hình ảnh gốc (Hình 1-12). Trong quá trình huấn luyện, các hình ảnh đã được che được sử dụng làm đầu vào cho mô hình, và các hình ảnh gốc được sử dụng làm nhãn.



Hình 1-12. Ví dụ về học tự giám sát: đầu vào (trái) và mục tiêu (phải)

Mô hình thu được có thể khá hữu ích, ví dụ như để sửa chữa ảnh bị hỏng hoặc xóa các đối tượng không mong muốn khỏi ảnh. Nhưng thường thì, việc huấn luyện một mô hình bằng phương pháp tự học có giám sát không phải là mục tiêu cuối cùng.

Thông thường, bạn muốn tinh chỉnh và điều chỉnh mô hình cho một nhiệm vụ hơi khác, một nhiệm vụ mà bạn thực sự quan tâm.

Ví dụ, giả sử điều bạn thực sự muốn là một mô hình phân loại thú cưng:

khi được cung cấp hình ảnh của bất kỳ thú cưng nào, mô hình sẽ cho bạn biết nó thuộc loài nào. Nếu bạn có một tập dữ liệu lớn gồm các bức ảnh thú cưng chưa được gắn nhãn, bạn có thể bắt đầu bằng cách huấn luyện một mô hình sửa chữa hình ảnh bằng cách sử dụng học tự giám sát. Khi mô hình hoạt động tốt, nó phải có khả năng phân biệt các loài thú cưng khác nhau: thực tế, khi nó sửa chữa một hình ảnh của một con mèo bị che mặt, nó biết rằng nó không được thêm khuôn mặt của một con chó. Giả sử kiến trúc của mô hình cho phép điều đó (và hầu hết các kiến trúc mạng nơ-ron đều cho phép), thì sau đó có thể điều chỉnh mô hình để nó dự đoán loài thú cưng thay vì sửa chữa hình ảnh. Bước cuối cùng bao gồm việc tinh chỉnh mô hình trên tập dữ liệu đã được gắn nhãn: mô hình đã biết hình dạng của mèo, chó và các loài vật nuôi khác, vì vậy bước này chỉ cần thiết để mô hình có thể học được mối liên hệ giữa các loài mà nó đã biết và các nhãn mà chúng ta mong đợi từ nó.

GHI CHÚ

Việc chuyển giao kiến thức từ nhiệm vụ này sang nhiệm vụ khác được gọi là học chuyển giao, và đây là một trong những kỹ thuật quan trọng nhất trong Học máy hiện nay, đặc biệt khi sử dụng mạng nơ-ron sâu (tức là mạng nơ-ron bao gồm nhiều lớp nơ-ron). Chúng ta sẽ thảo luận chi tiết về điều này trong [Phần II](#).

Một số người coi học tự giám sát là một phần của học không giám sát, vì nó xử lý các tập dữ liệu hoàn toàn không được gắn nhãn. Nhưng học tự giám sát sử dụng các nhãn (được tạo ra) trong quá trình huấn luyện, vì vậy về mặt đó, nó gần với học có giám sát hơn. Và thuật ngữ "học không giám sát" thường được sử dụng khi xử lý các tác vụ như phân cụm, giảm chiều dữ liệu hoặc phát hiện bất thường, trong khi học tự giám sát tập trung vào cùng một mục tiêu đó.

Các nhiệm vụ được coi là học có giám sát: chủ yếu là phân loại và hồi quy.
Tóm lại, tốt nhất nên xem học tự giám sát là một loại riêng biệt.

Học tăng cường

Học tăng cường là một lĩnh vực hoàn toàn khác. Hệ thống học tập, được gọi là tác nhân trong ngữ cảnh này, có thể quan sát môi trường, lựa chọn và thực hiện các hành động, và nhận được phần thưởng (hoặc hình phạt dưới dạng phần thưởng âm, như được minh họa trong **Hình 1-13**). Sau đó, nó phải tự học chiến lược tốt nhất, được gọi là chính sách, để nhận được phần thưởng cao nhất theo thời gian. Chính sách xác định hành động mà tác nhân nên chọn khi ở trong một tình huống nhất định.